

Designing a multi-language live programming tool with Phoenix and GenStage

Andrea Amantini

 lo_zampino

 zampino

nextjournal

Designing a multi-language live programming tool with Phoenix and GenStage*

Andrea Amantini

 lo_zampino

 zampino

nextjournal

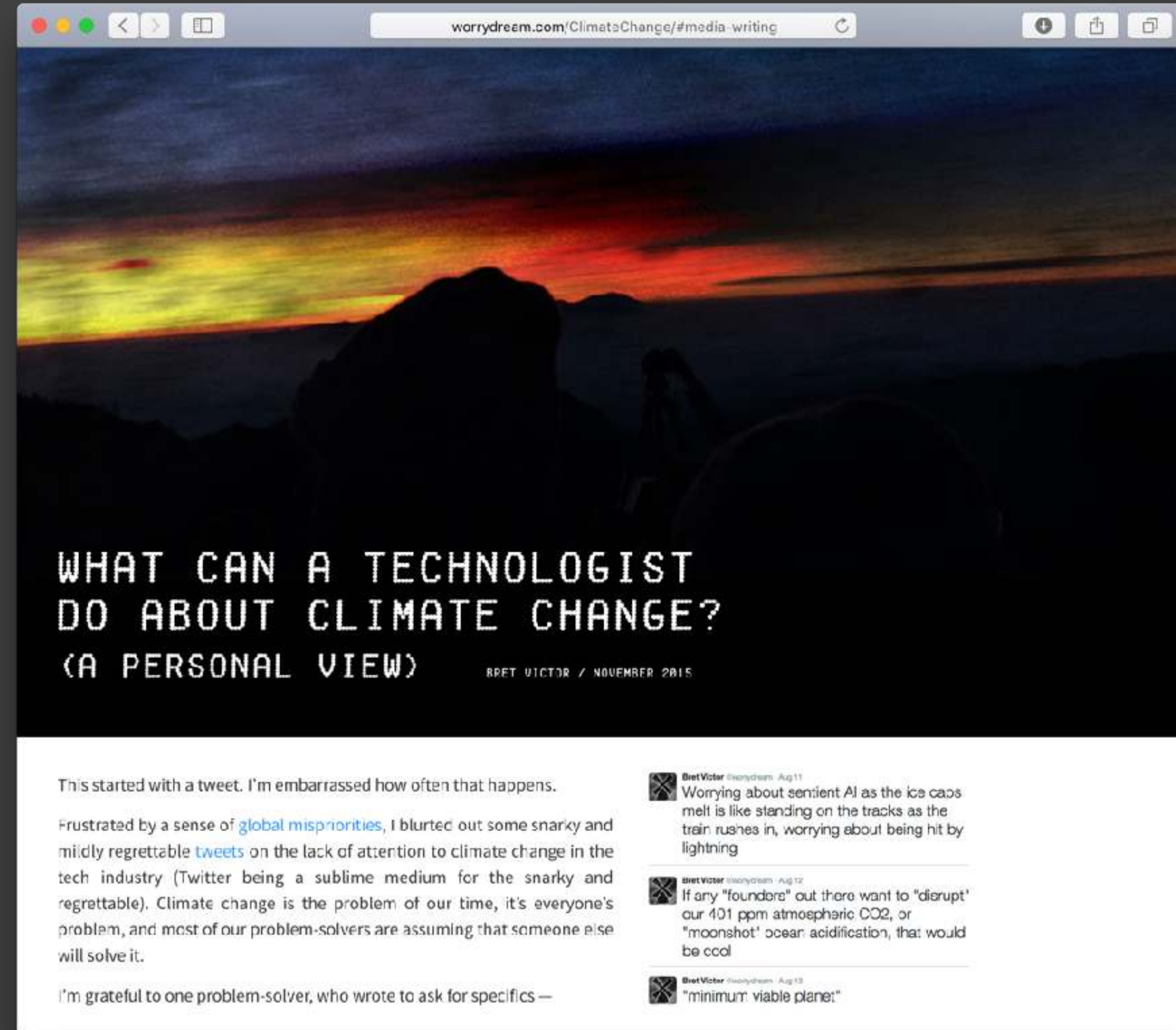
*and why you don't want to use it this way

“Imagine an authoring tool designed for arguing from evidence...

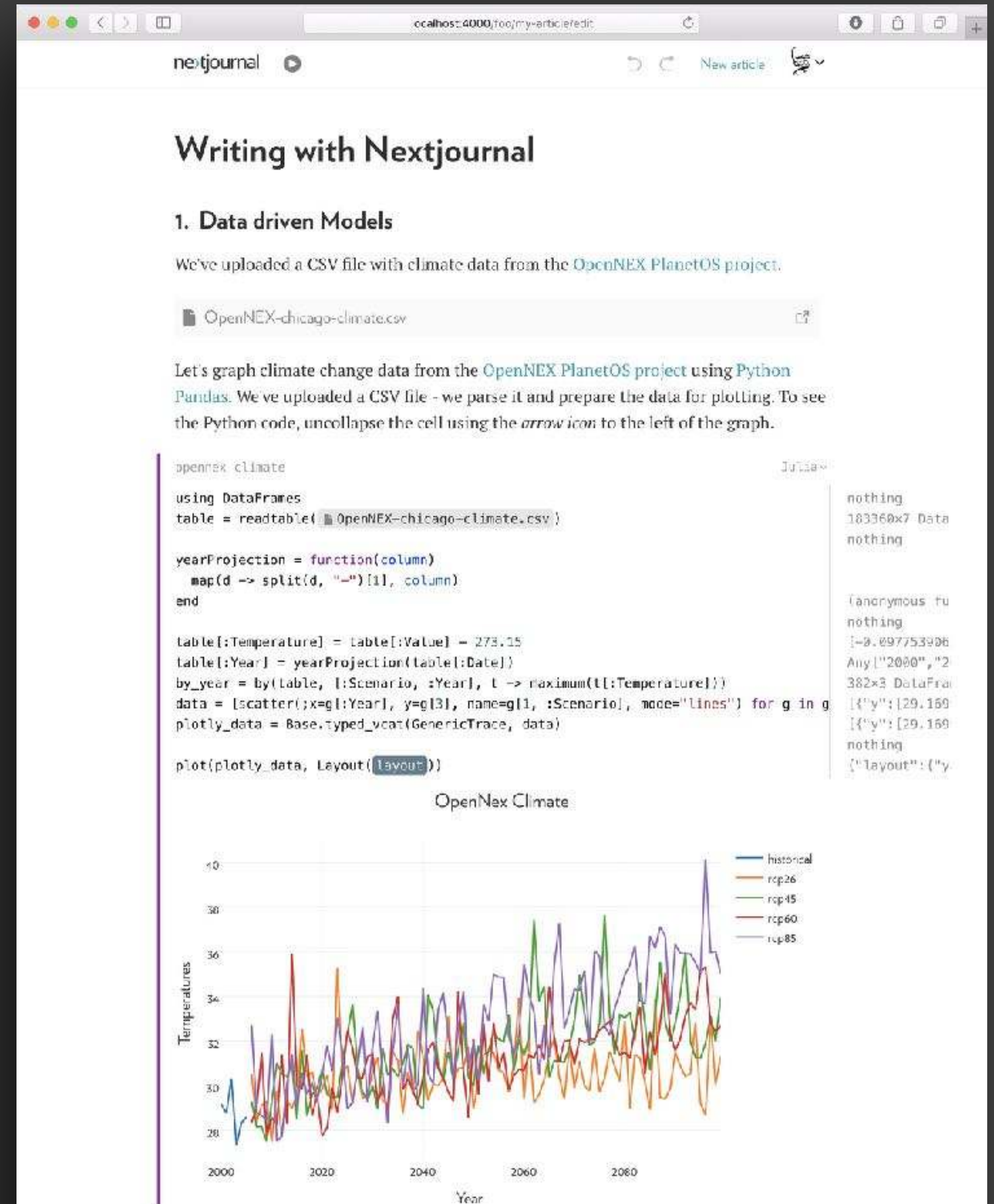
...literally *autocompleting* sourced facts directly into the document...

...what if it were as easy to insert facts, data, and models as it is to insert emoji and cat photos?”

—Bret Victor
worrydream.com/ClimateChange



nextjournal





I Title



>

↶

↷

↺

↻

Runner

✓

everything done

Welcome to nextjournal

A Nextjournal article is composed of cells. This one is a normal paragraph.
Hovering between sections, we can add new cells. Code cells can run Python, R or Julia... live!

Insert

Search

Code Cell: Bash

Code Cell: Julia

Code Cell: Python

Code Cell: R

Code Listing

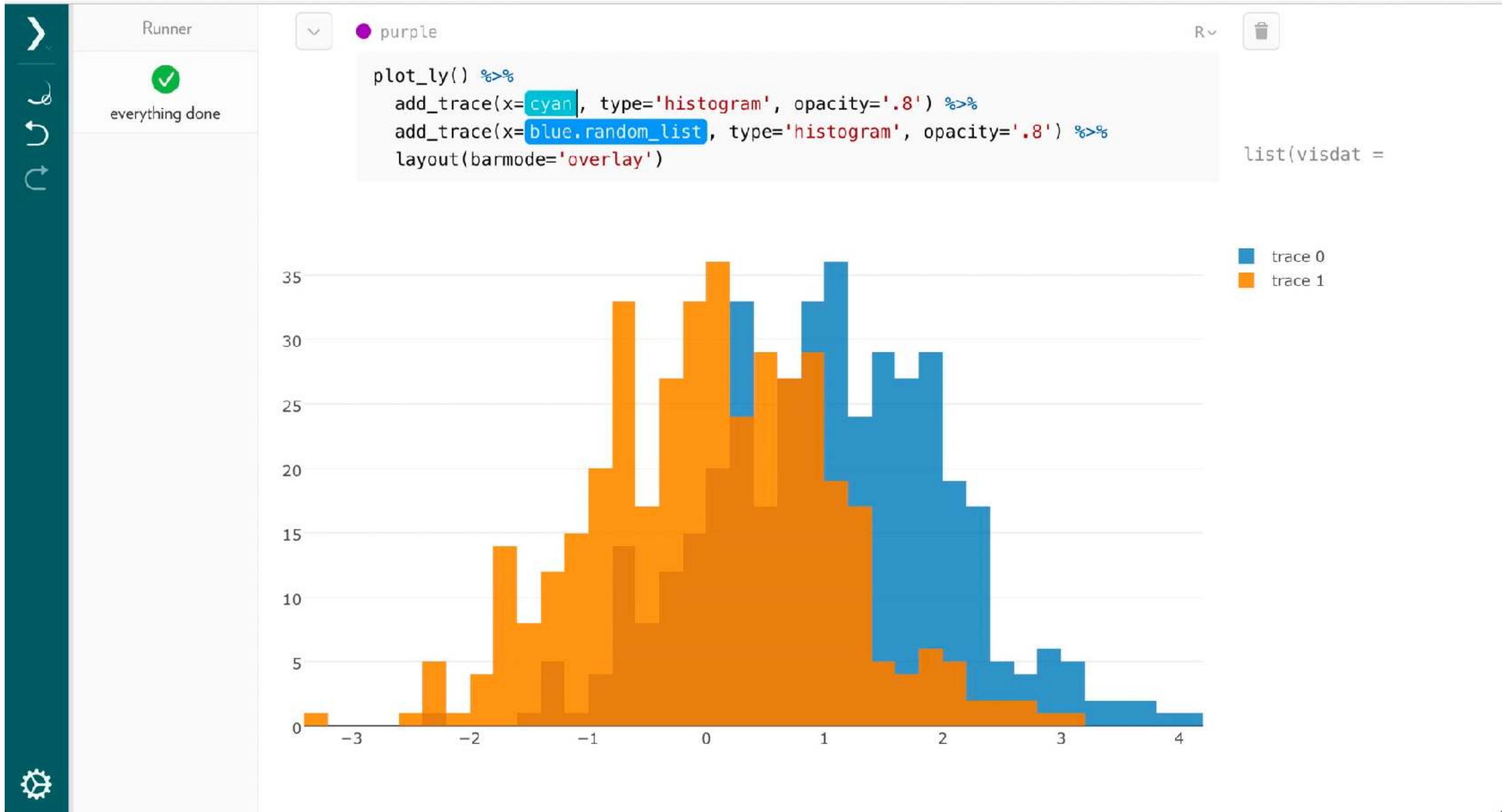
File

Formula

Image

Julia

400
[-0.641151916
nice!





● Runner ready



functions

Cell has changes



Alanine Dipeptide Analysis

1. Introduction

We are going to analyze NVE simulations of terminally blocked *Alanine* a little 22-atom compound often used the simplest molecular system to be studied and analyzed. In some sense this is the hydrogen atom of the biomolecular simulation community. For the view of a biologist this is the simplest peptide you could build (although it does not exist in nature). It is comprised of one amino acid *Alanine* with some initial and final atoms. In reality proteins are built out of chains of the (20 natural occurring) amino acids.

[1] J. D. Chodera, W. C. Swope, J. W. Pitera, and K. A. Dill, “*Long-time protein folding dynamics from short-time molecular dynamics simulations*,” *Multiscale Model. Sim.*, vol. 5, no. 4, pp. 1214–1226, 2006.

2. File imports

the file uploaded contains the so called replica-exchange trajectories

>

functions

Cell has changes

↺

↻

⚙️

● Runner ready

Bash

Docker Image: nextjournal/bash:tag

Preamble: none ▾

Julia

Docker Image: nextjournal/julia:tag

Preamble: none ▾

Python

Docker Image: nextjournal/python-pyemma:35c8d0a

Preamble: ● preamble ▾

R

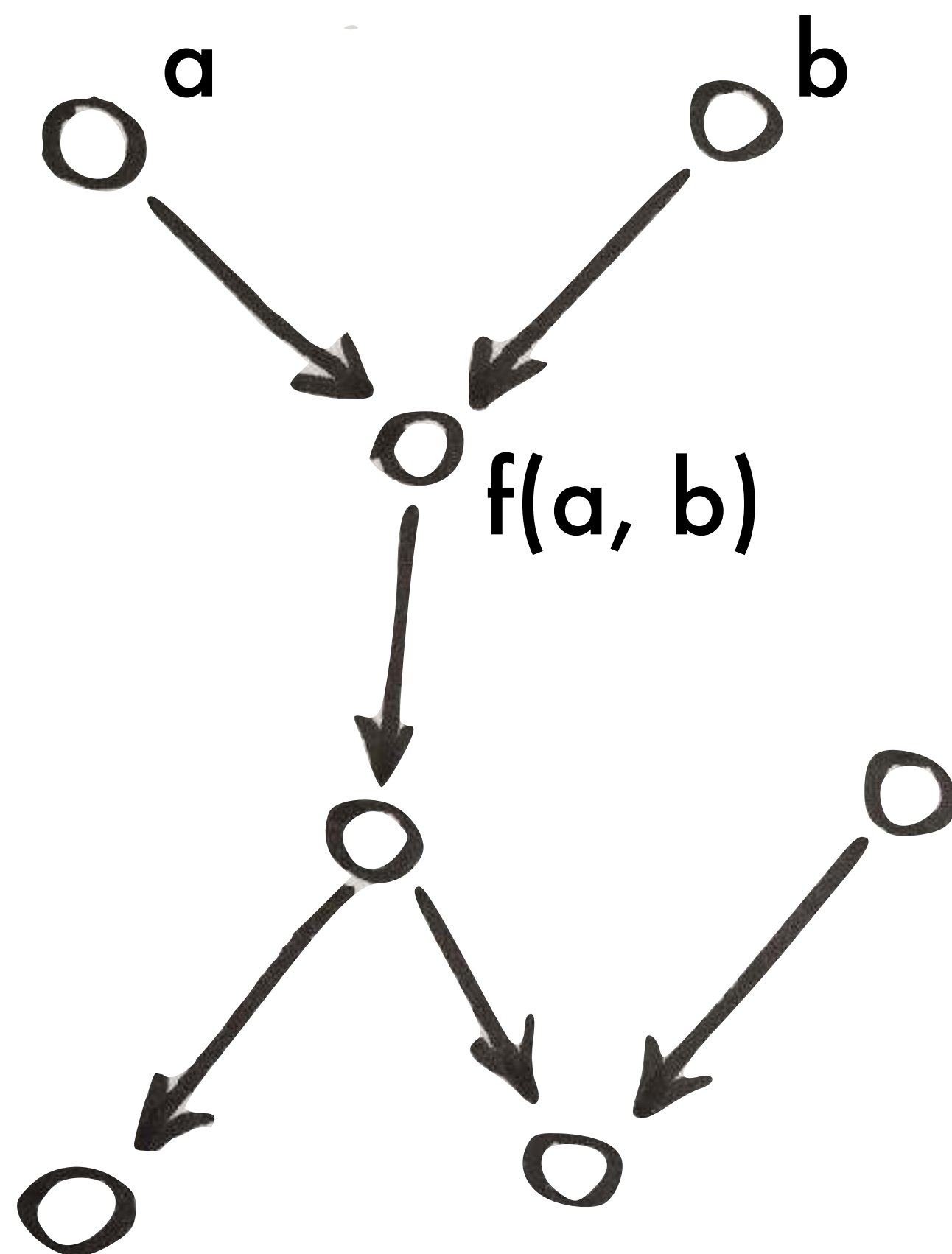
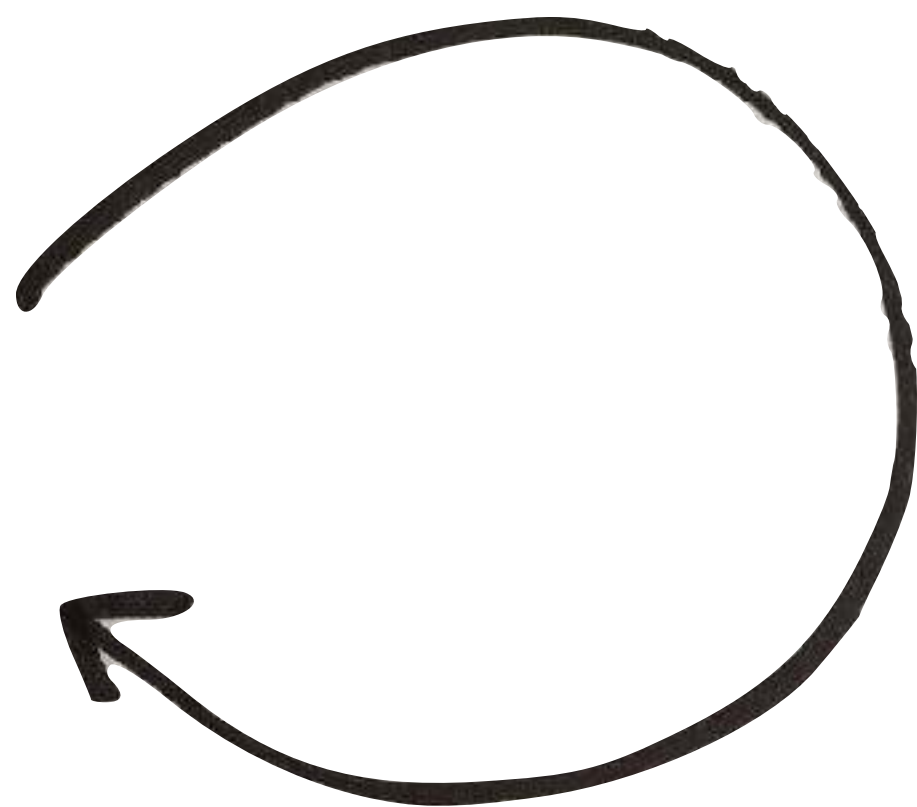
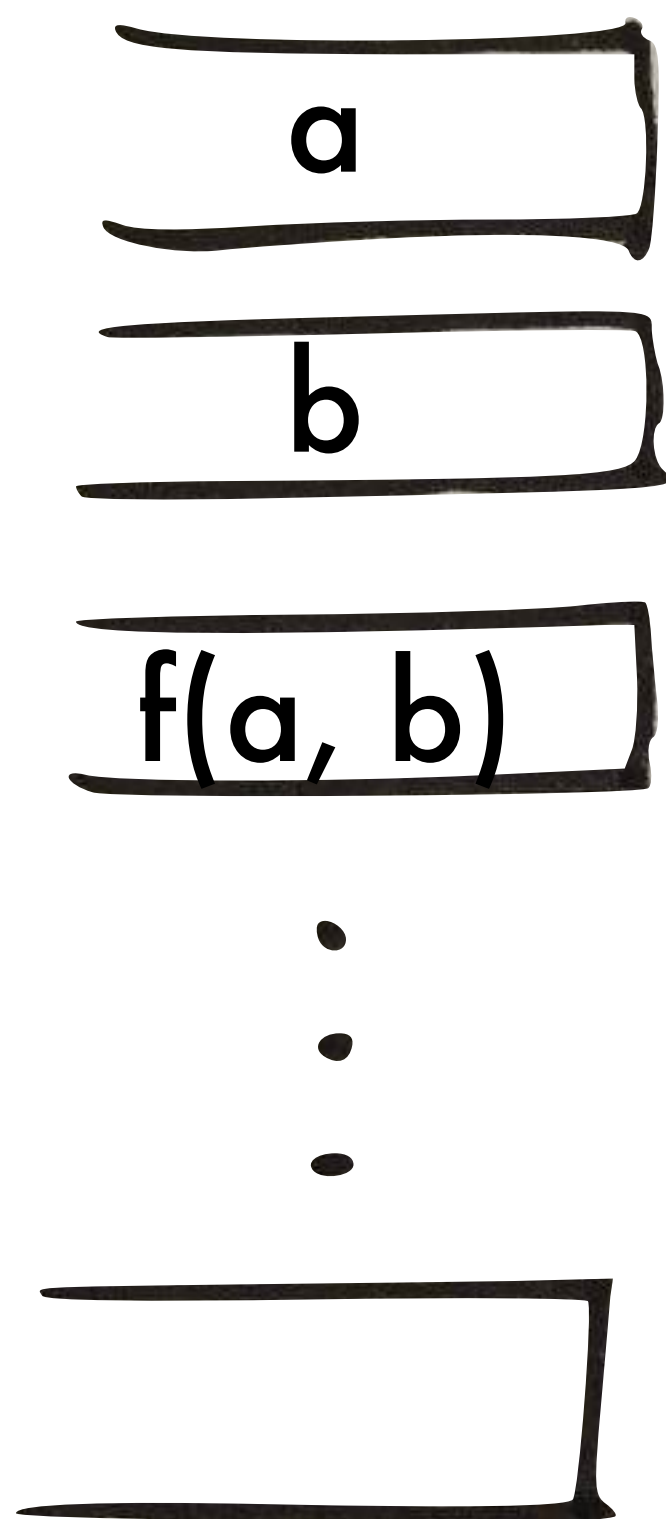
Docker Image: nextjournal/r:tag

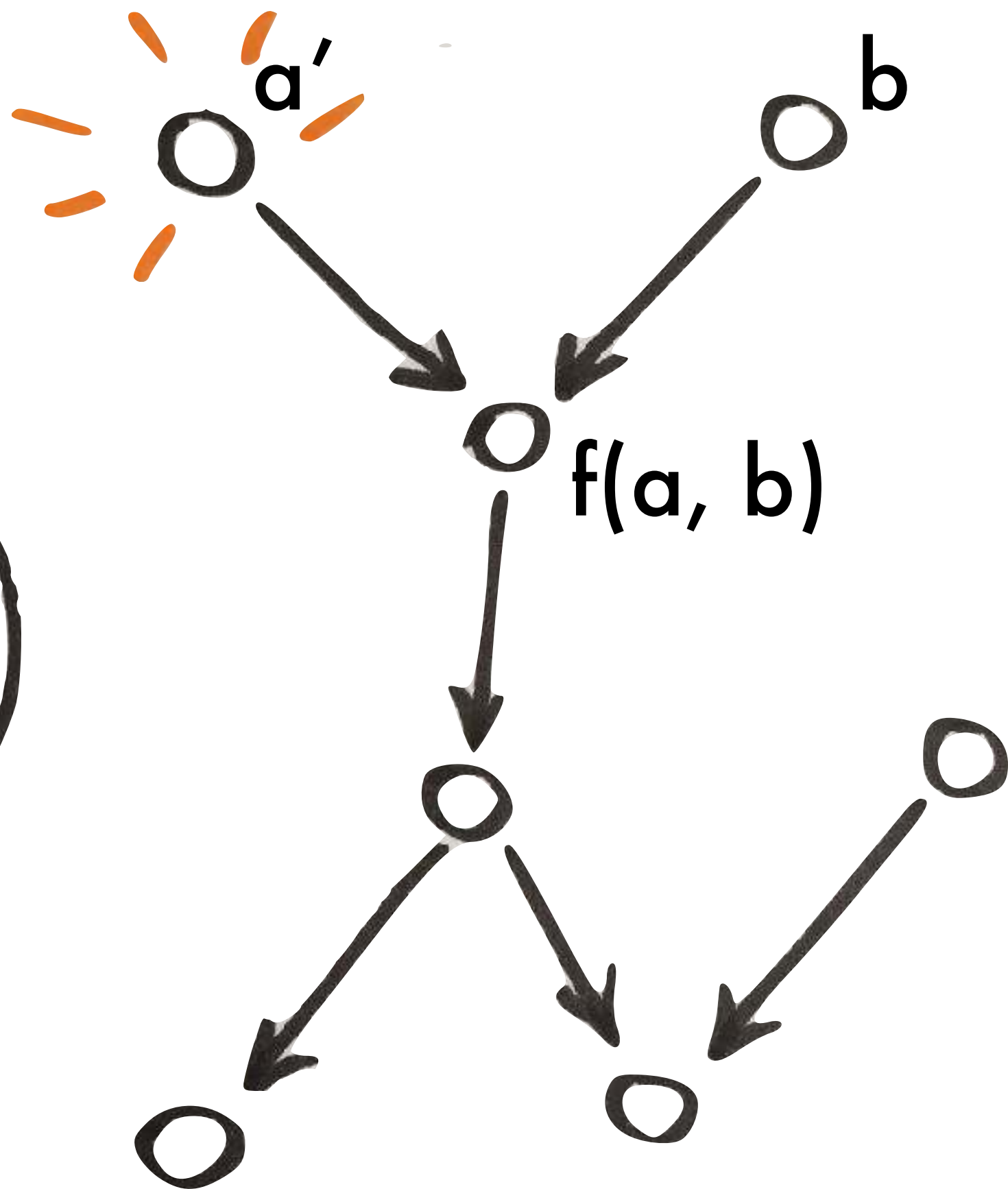
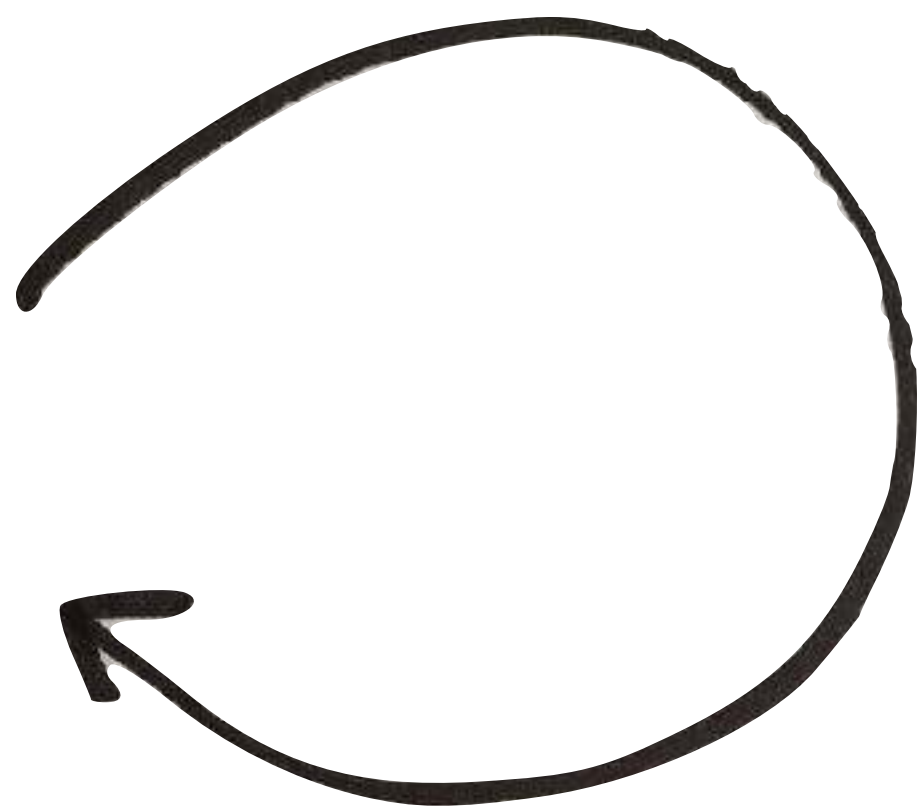
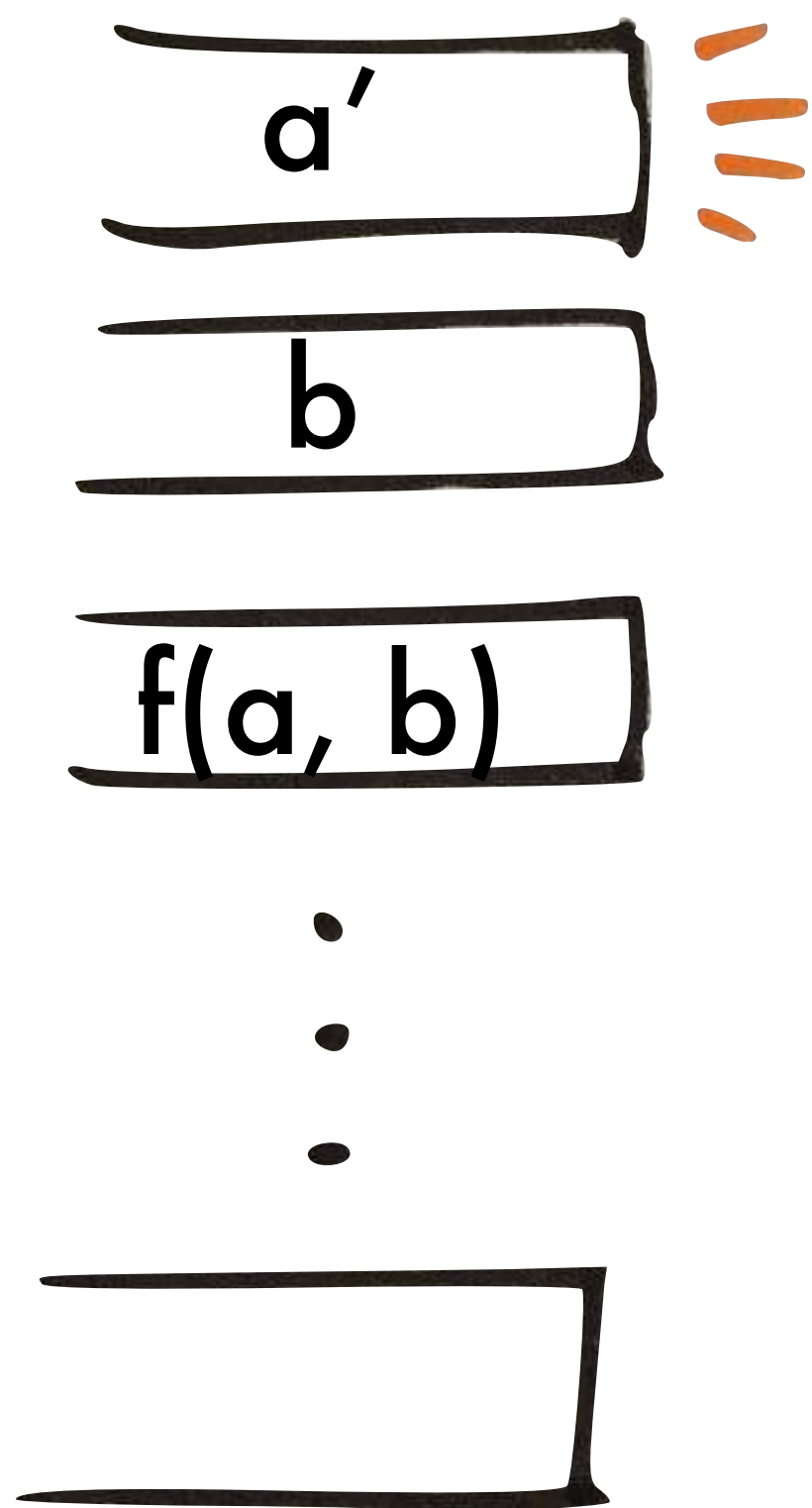
Preamble: none ▾

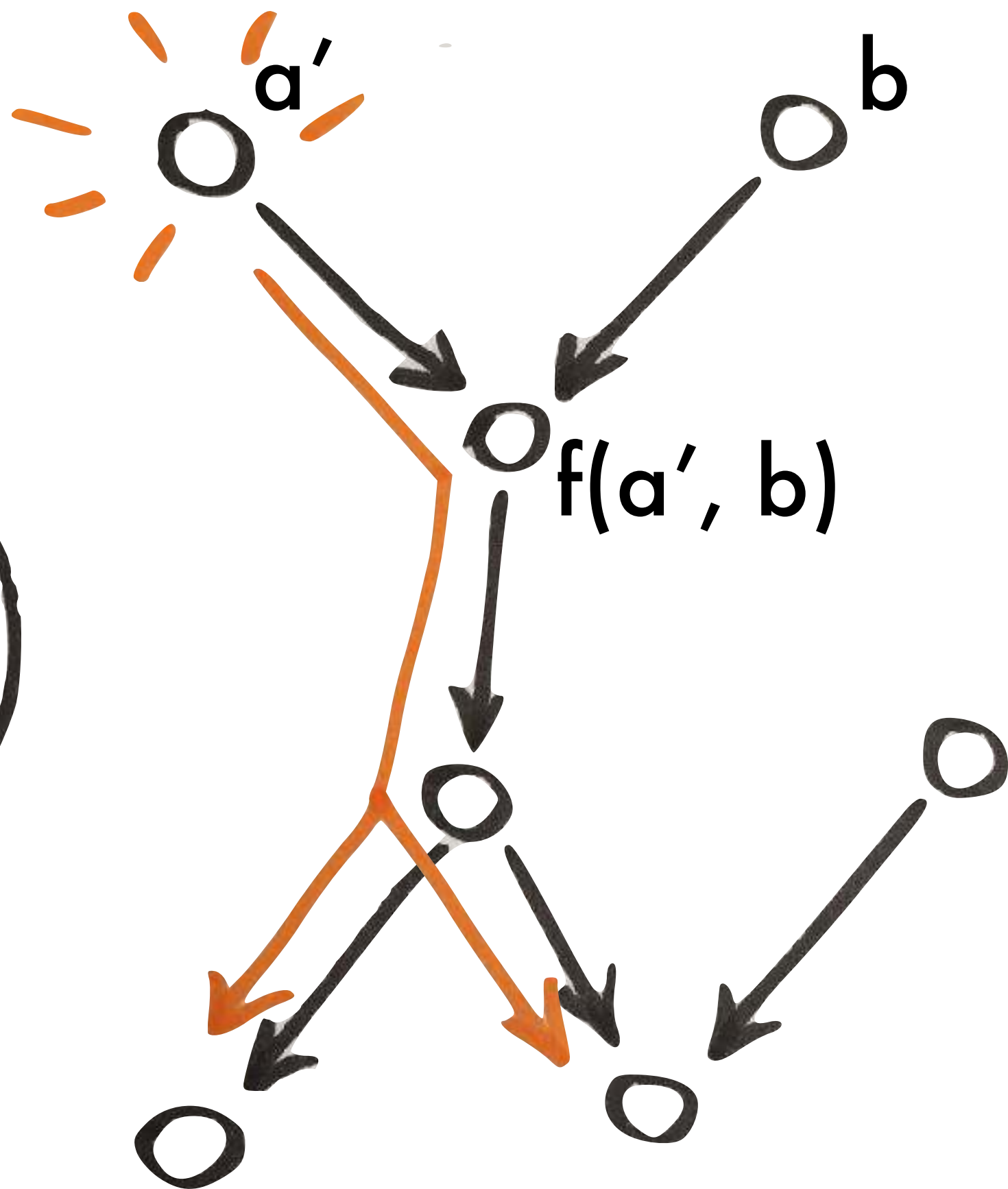
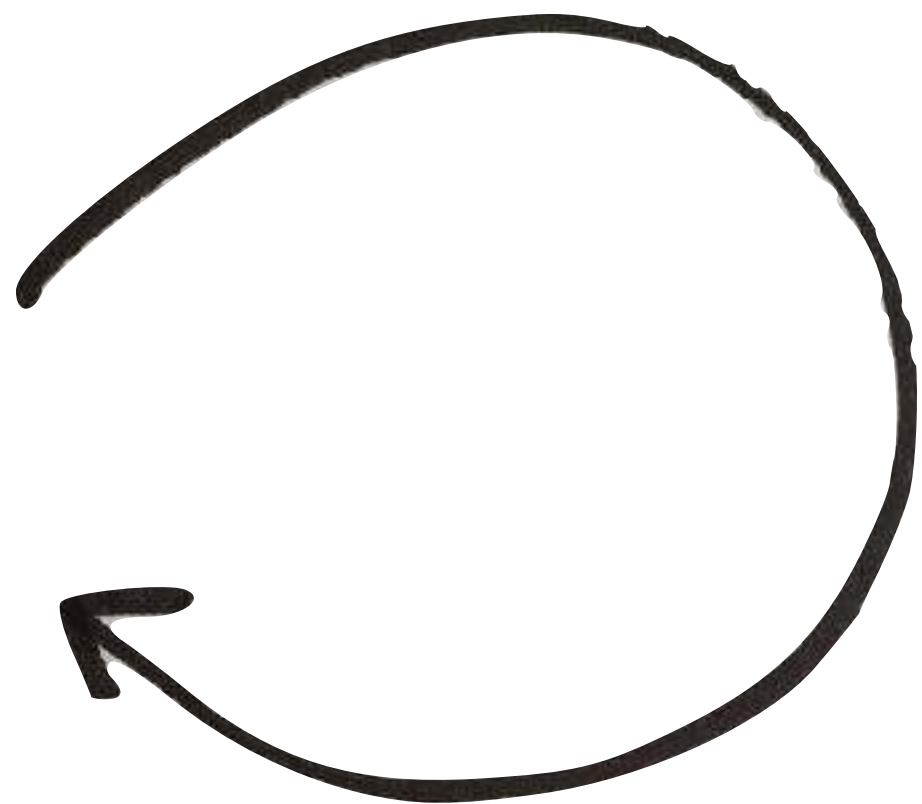
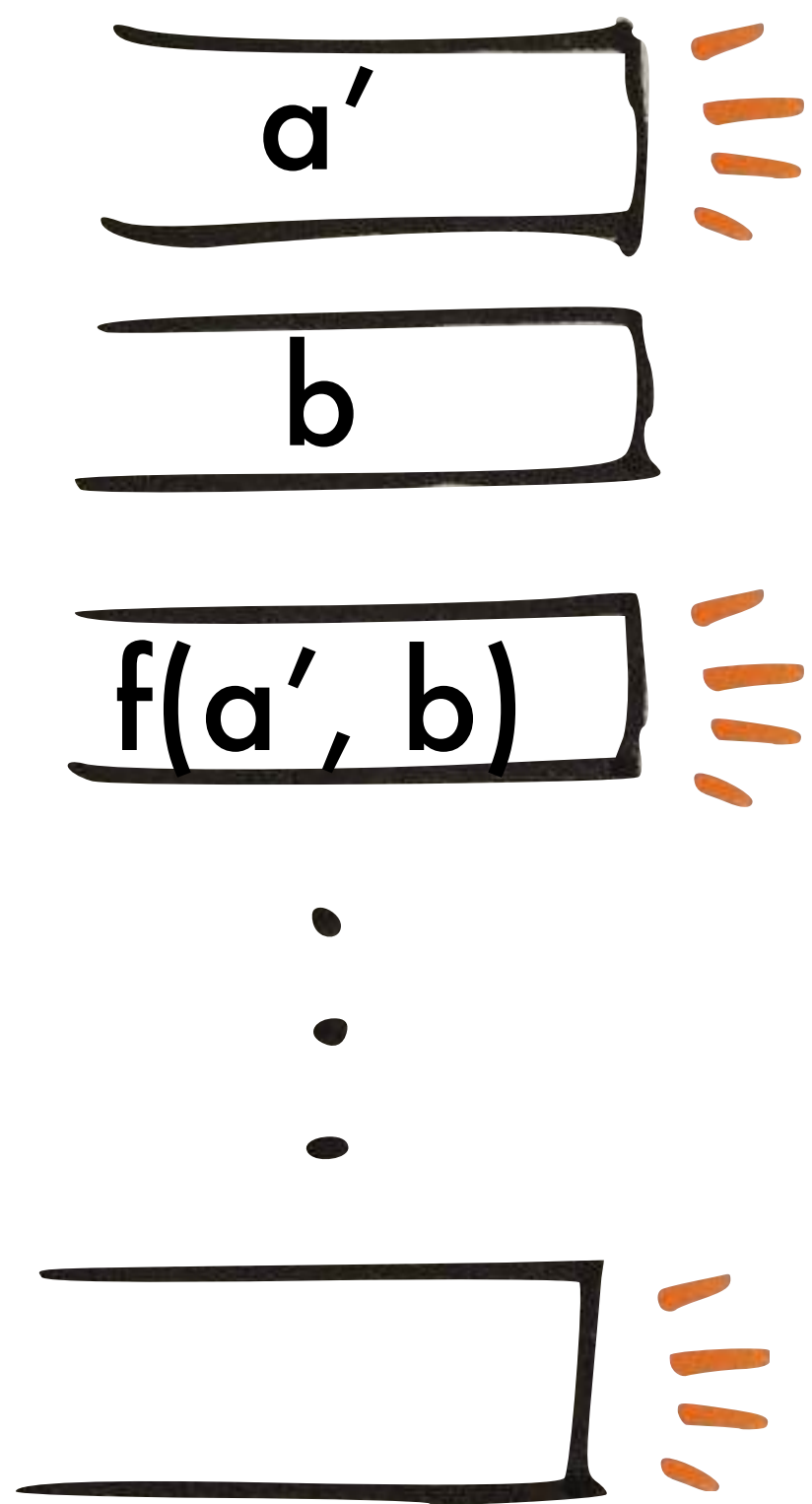
Done

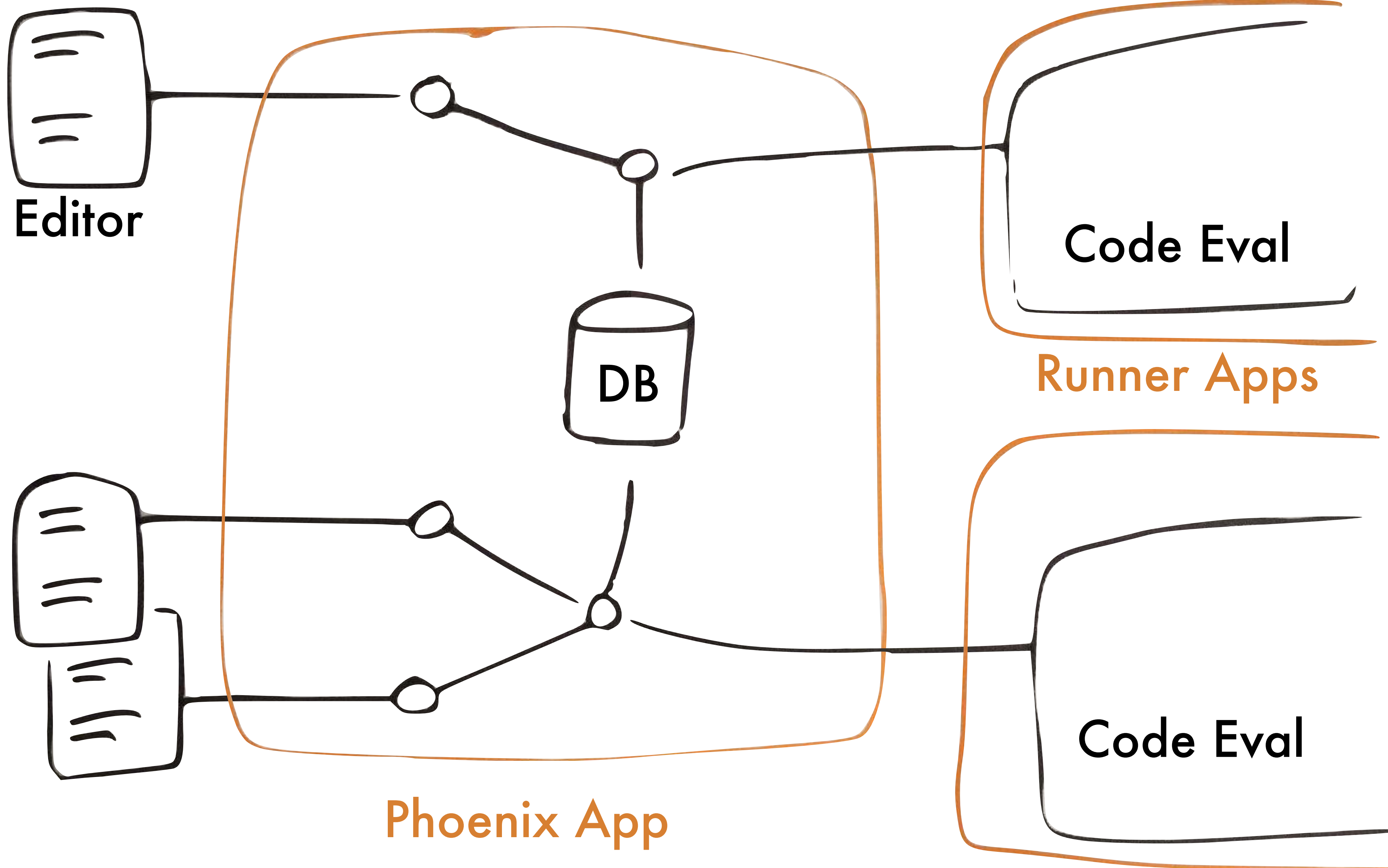
1. Introduction

We are going to analyze NVE simulations of terminally blocked *Alanine* a little 22-atom compound often used the simplest molecular sytem to be studied and anayzed. In some sense this is the hydrogen atom of the biomolecular simulation

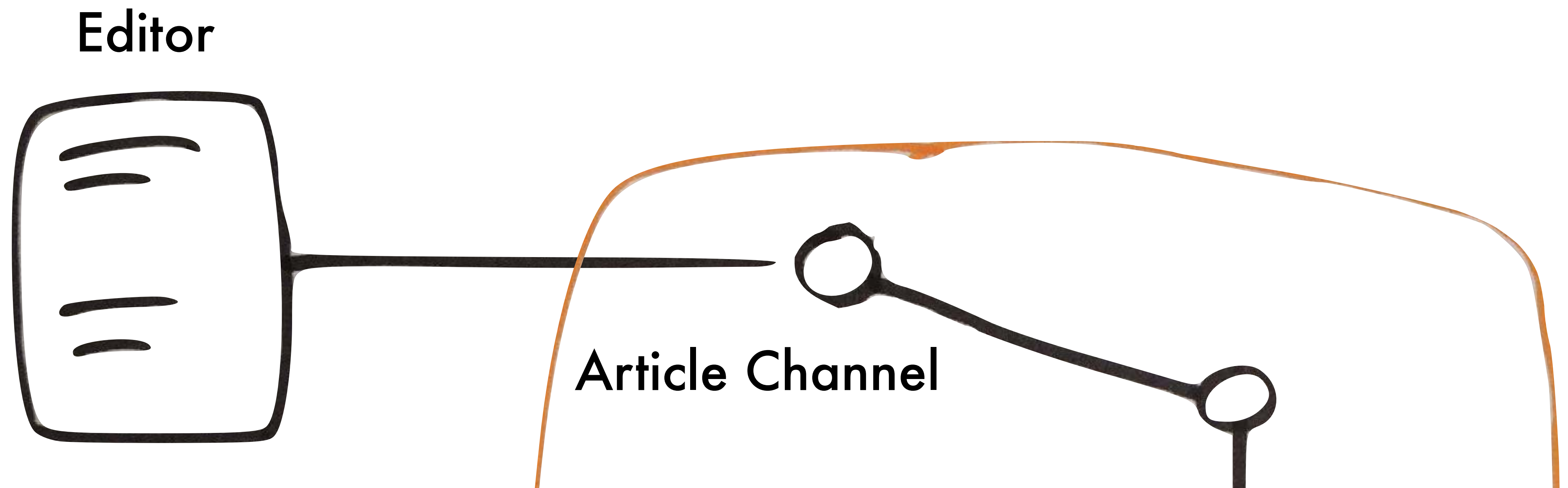




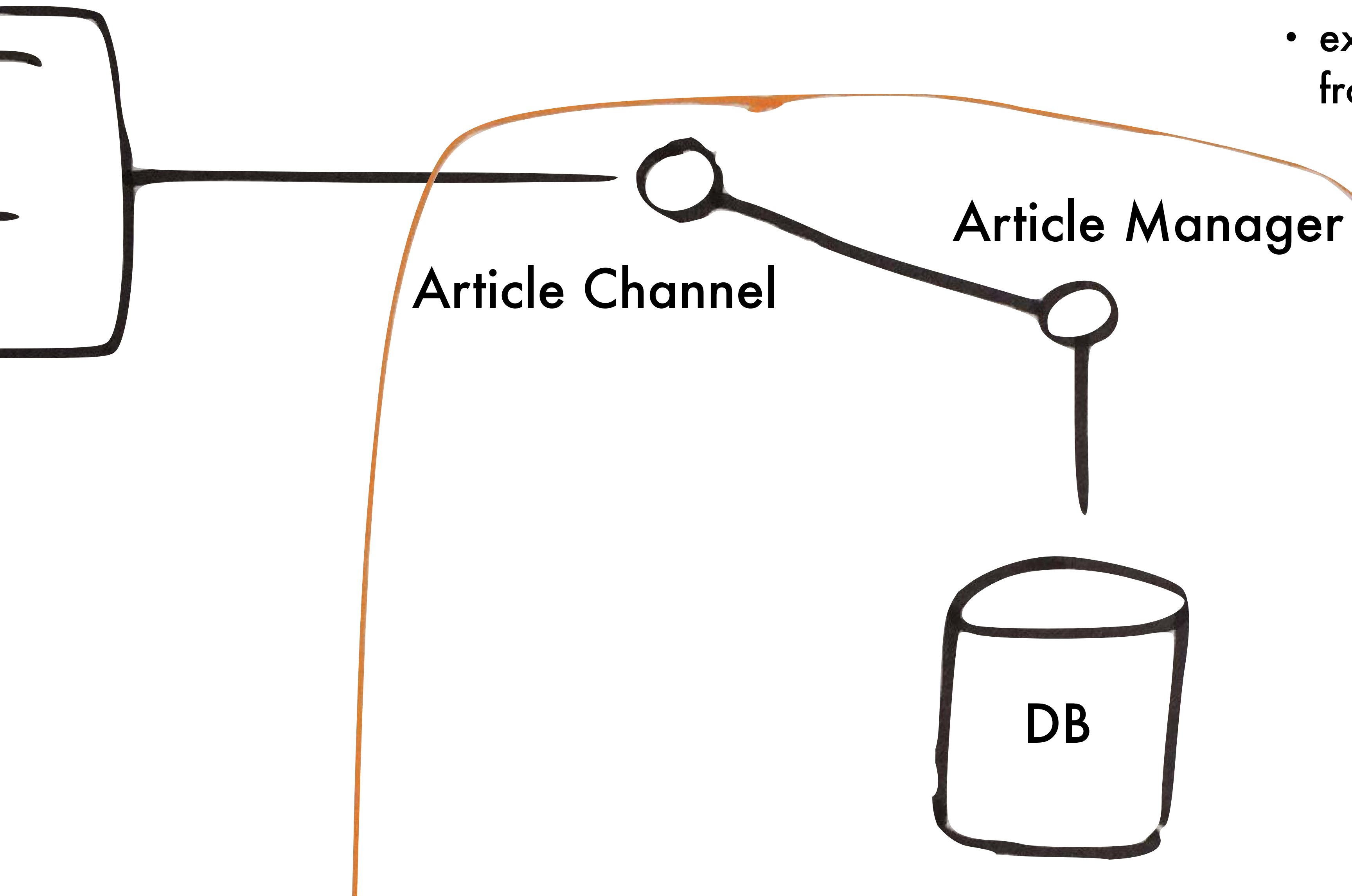




- ClojureScript & re-frame
- Transit Format – Websocket
Serialiser/Ecto Type
- exchange clojure/Elixir terms
(keywords $\Rightarrow$ atoms)
- JSON (or MsgPack) based

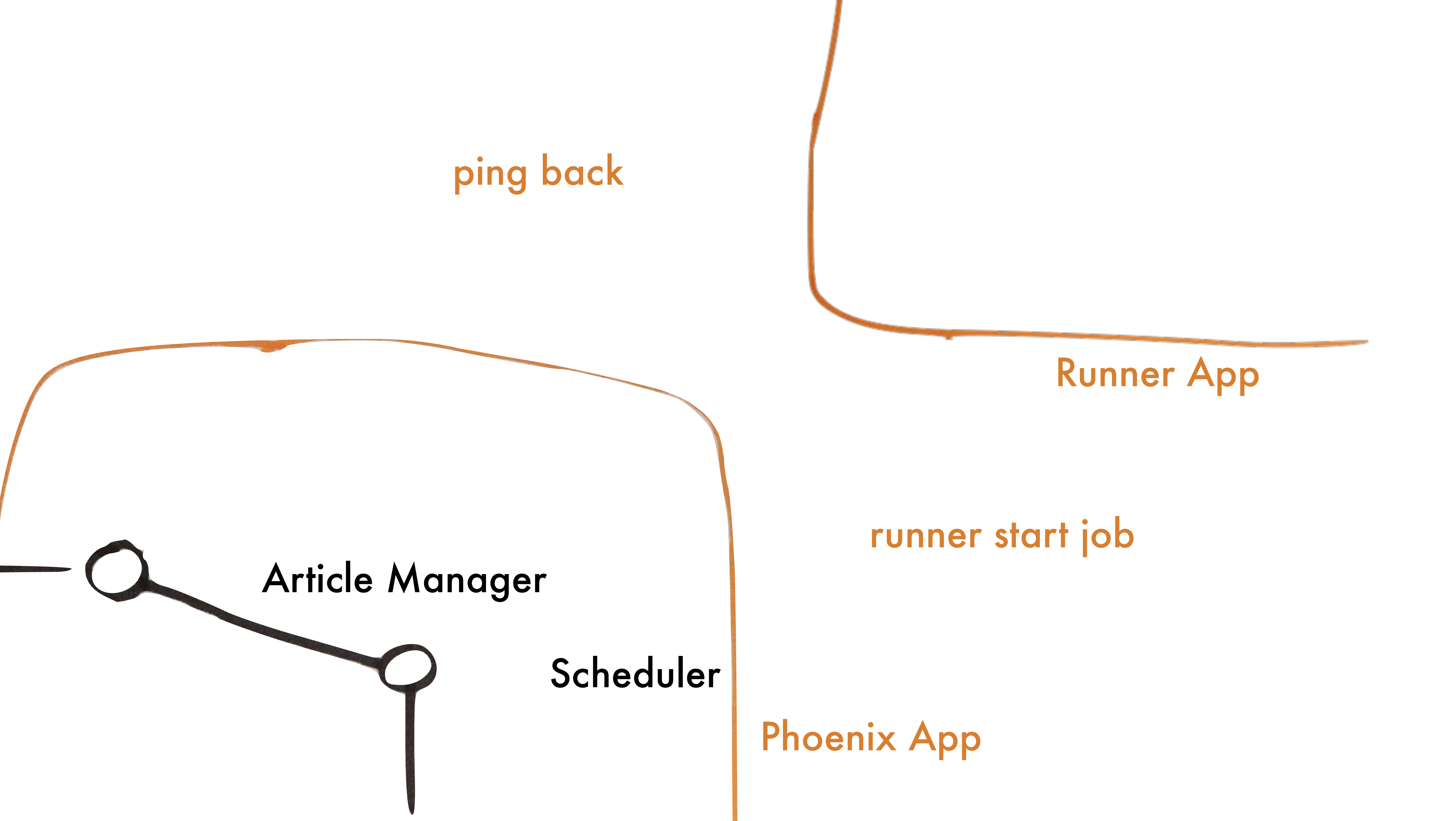


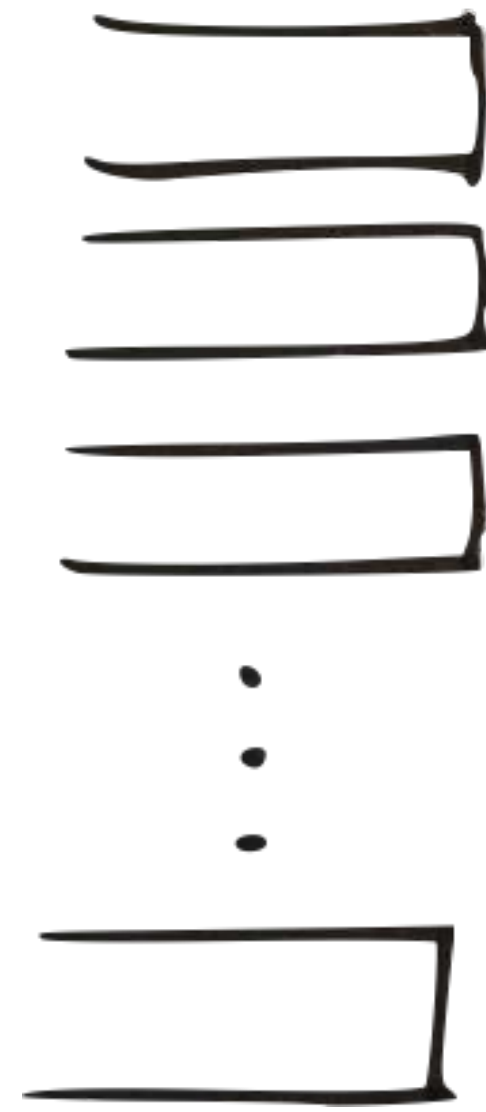
or



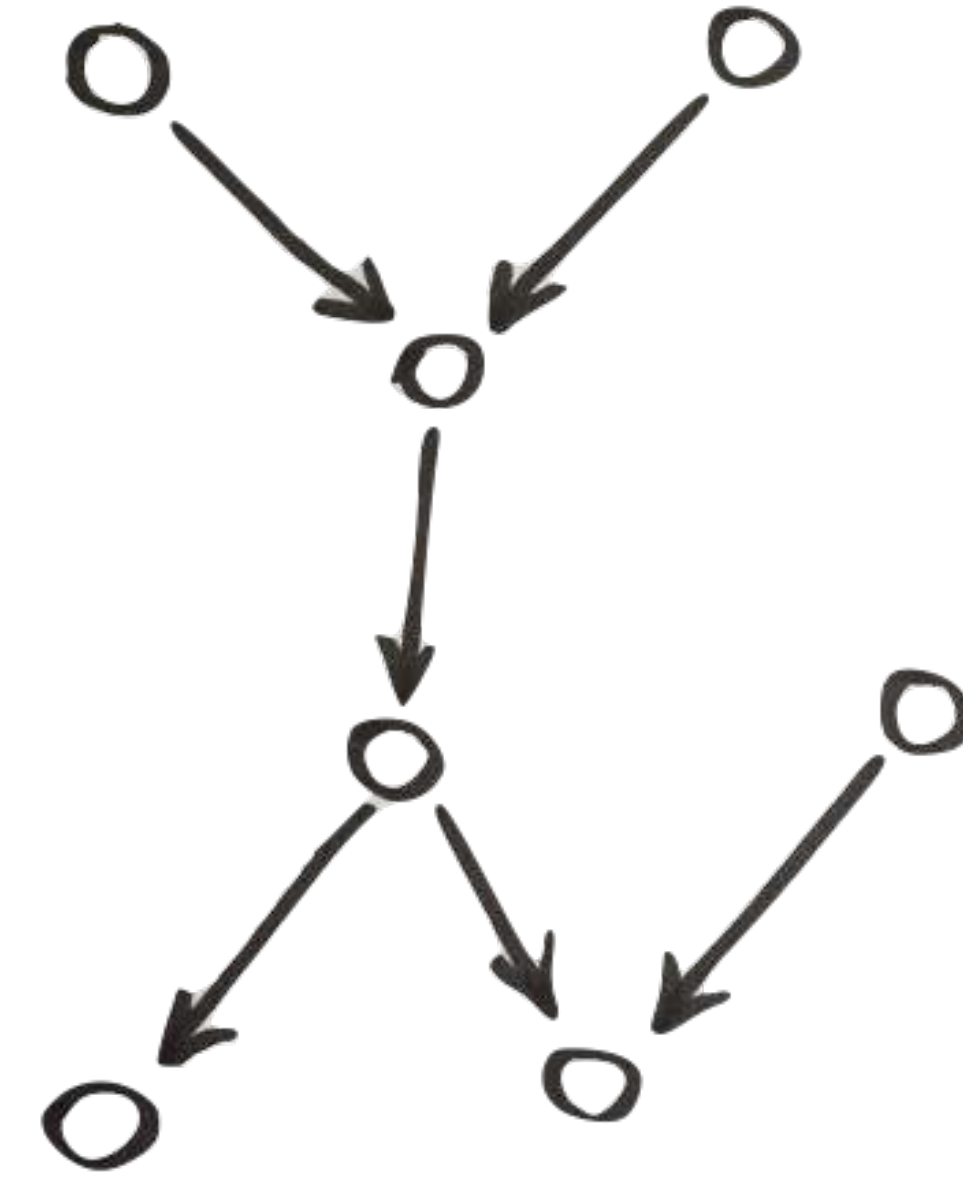
- Persists Edit Operations
serialising writes to DB

- extracts CodeCells
from Article State

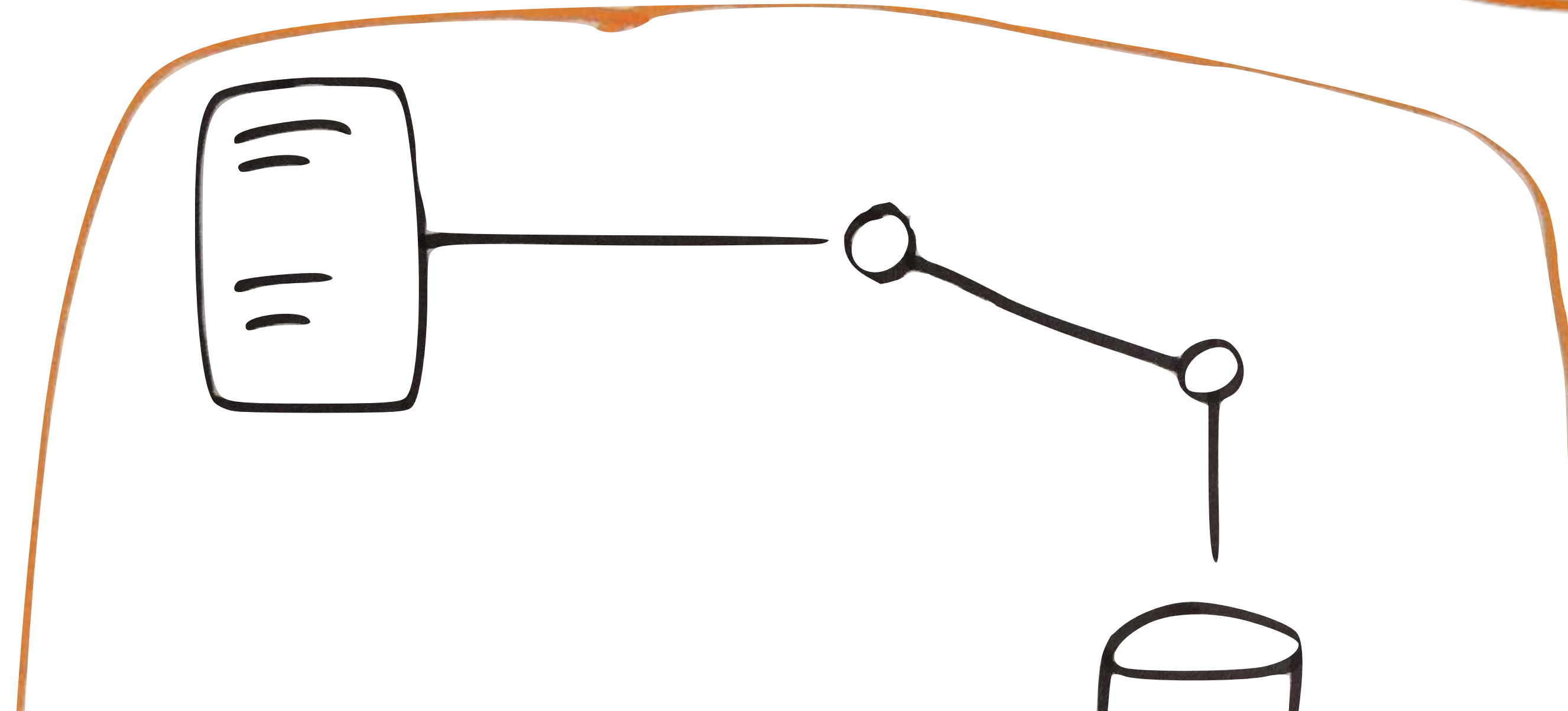




Editor

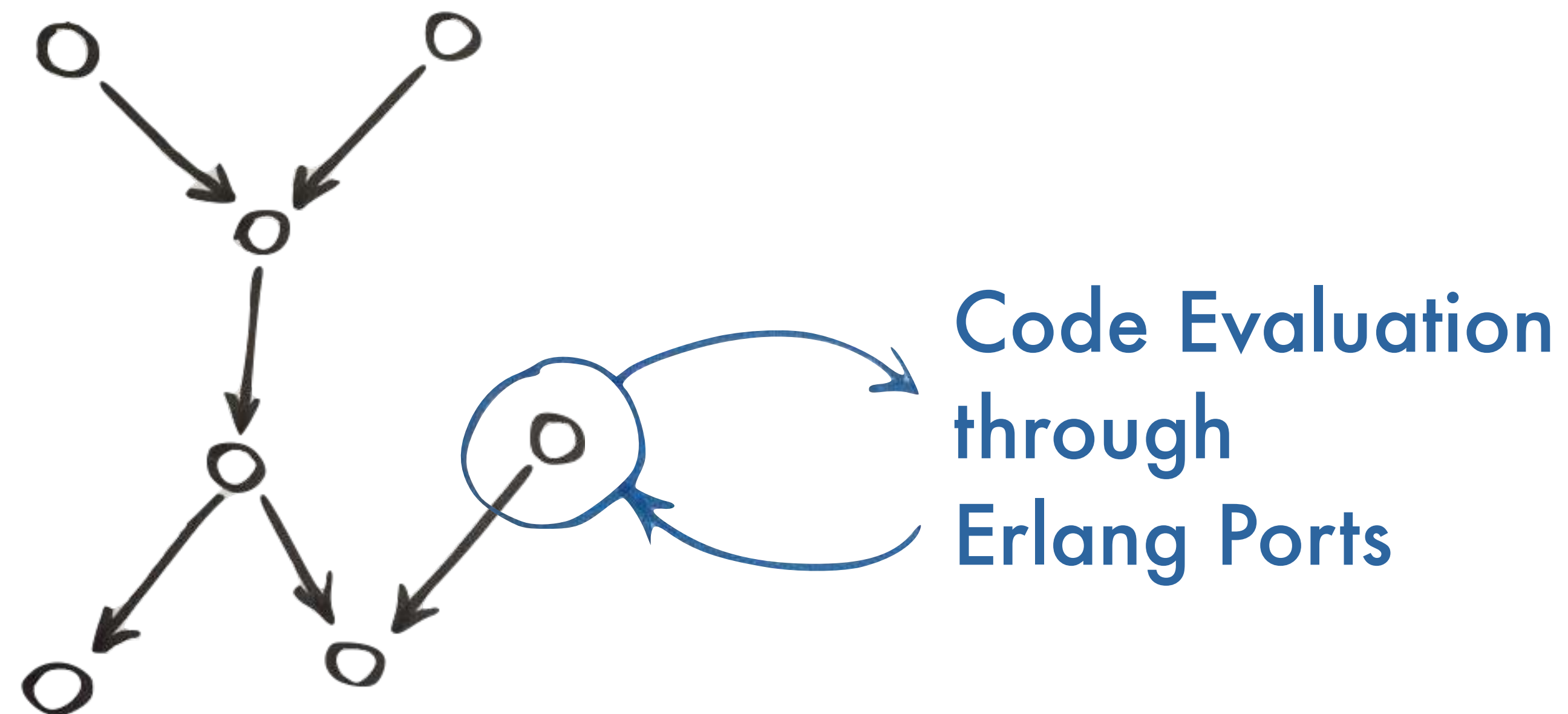


Runner App



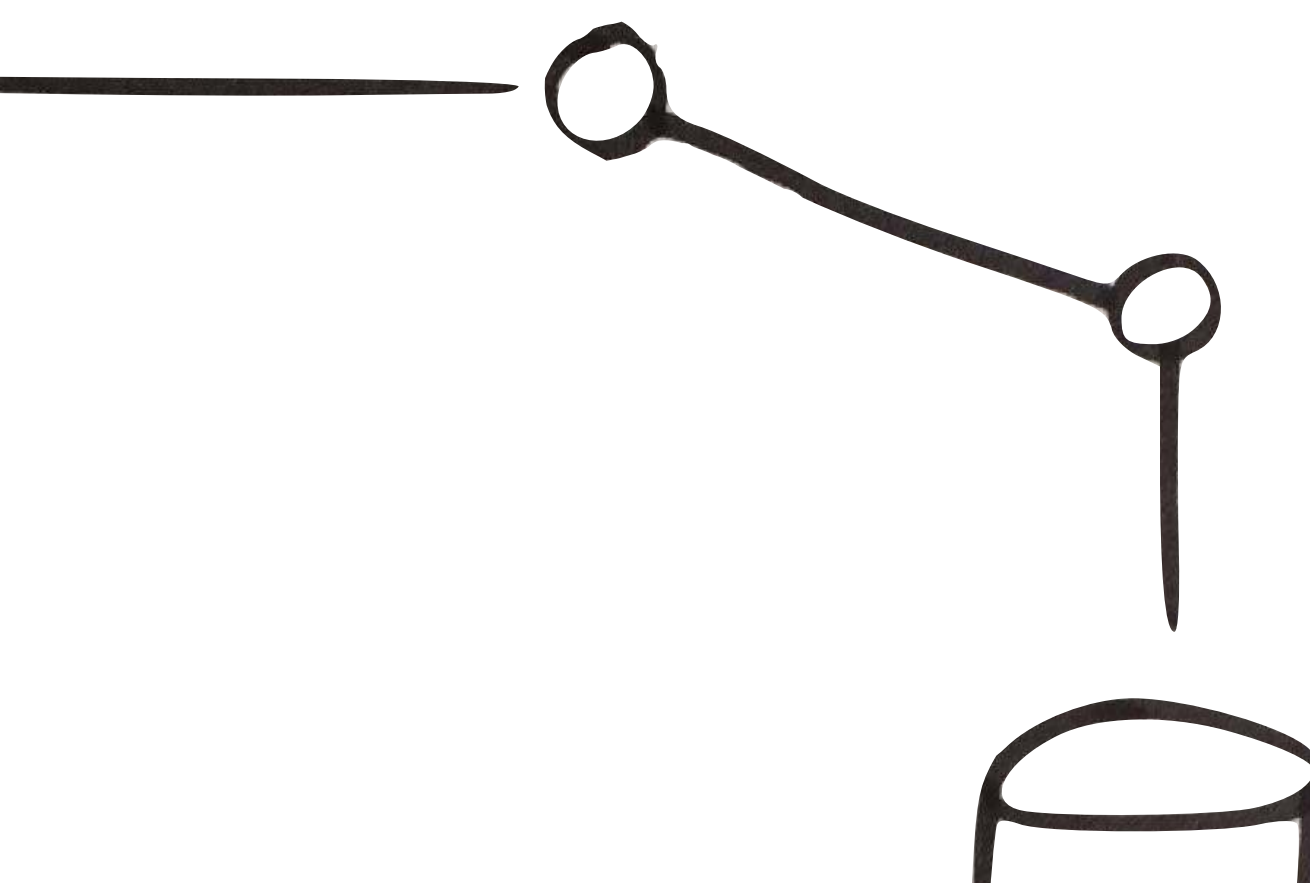
Phoenix App

Phoenix PubSub



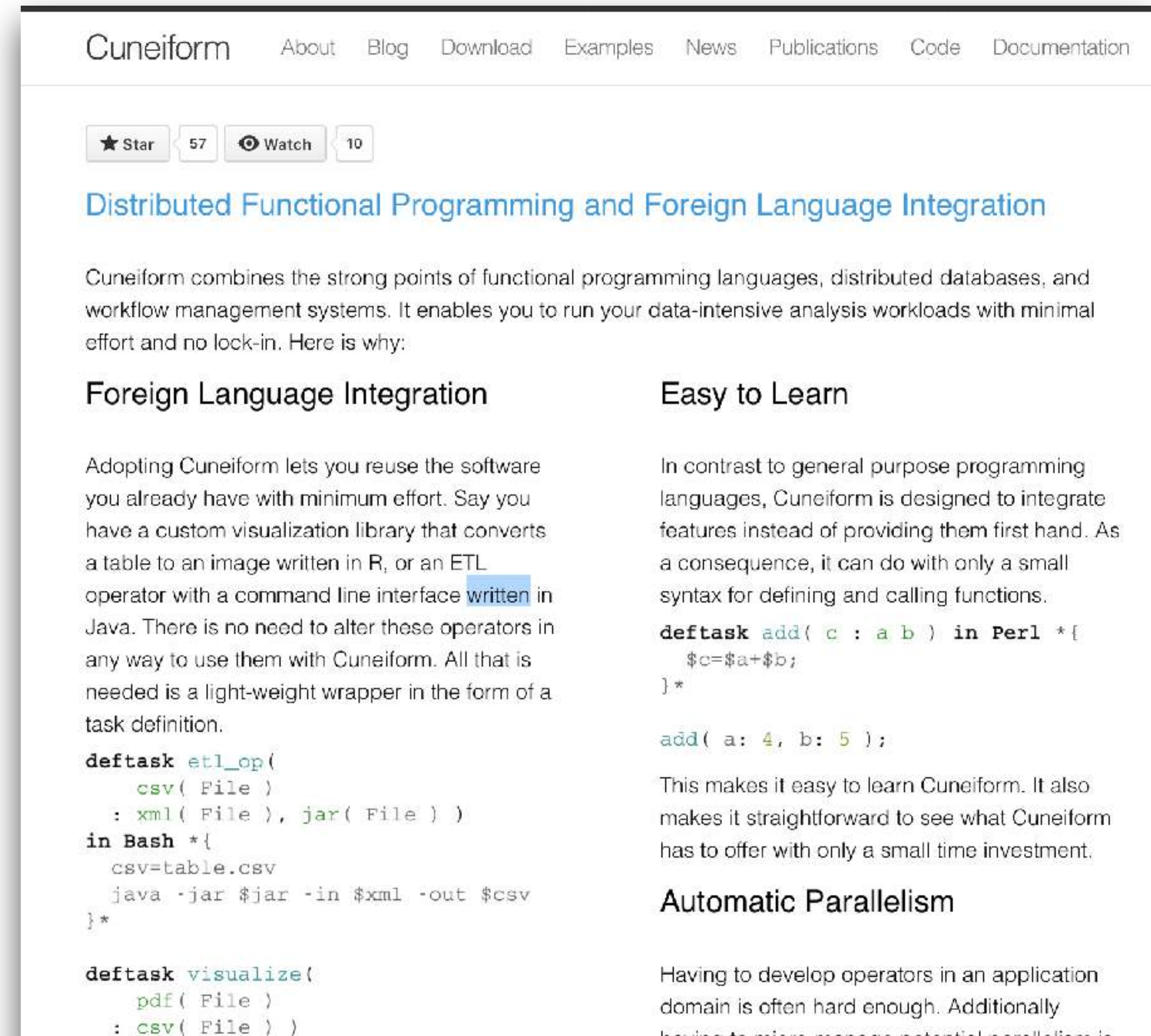
Runner App

Phoenix App



Take I: Cuneiform backend

- Jörgen Brandt's (@joergenbr) cuneiform-lang.org
- *erlang* based language to abstract foreign language interaction and parallelisation
- Query-based graph of tasks



The screenshot shows the GitHub repository page for Cuneiform. The repository name 'Cuneiform' is at the top left of the page header. Navigation links include 'About', 'Blog', 'Download', 'Examples', 'News', 'Publications', 'Code', and 'Documentation'. Below the header, there are buttons for 'Star' (57), 'Watch' (10), and 'Fork' (10). The repository description is 'Distributed Functional Programming and Foreign Language Integration'. A paragraph explains that Cuneiform combines functional programming, distributed databases, and workflow management. The page is divided into two columns. The left column, titled 'Foreign Language Integration', describes how Cuneiform allows reusing existing software by wrapping foreign language operators. It includes a code snippet for a task named 'etl_op' that uses 'csv', 'xml', and 'jar' operators in a Bash shell. The right column, titled 'Easy to Learn', contrasts Cuneiform with general-purpose languages and shows a simple 'deftask add' in Perl. Below this, it states that Cuneiform is easy to learn and straightforward to use. A third section, 'Automatic Parallelism', is partially visible at the bottom right, mentioning the difficulty of developing operators and the potential for parallelism.

Cuneiform

About Blog Download Examples News Publications Code Documentation

★ Star 57 Watch 10 Fork 10

Distributed Functional Programming and Foreign Language Integration

Cuneiform combines the strong points of functional programming languages, distributed databases, and workflow management systems. It enables you to run your data-intensive analysis workloads with minimal effort and no lock-in. Here is why:

Foreign Language Integration

Adopting Cuneiform lets you reuse the software you already have with minimum effort. Say you have a custom visualization library that converts a table to an image written in R, or an ETL operator with a command line interface written in Java. There is no need to alter these operators in any way to use them with Cuneiform. All that is needed is a light-weight wrapper in the form of a task definition.

```
deftask etl_op(  
  csv( File )  
  : xml( File ), jar( File ) )  
in Bash *{  
  csv=table.csv  
  java -jar $jar -in $xml -out $csv  
}*  
  
deftask visualize(  
  pdf( File )  
  : csv( File ) )
```

Easy to Learn

In contrast to general purpose programming languages, Cuneiform is designed to integrate features instead of providing them first hand. As a consequence, it can do with only a small syntax for defining and calling functions.

```
deftask add( c : a b ) in Perl *{  
  $c=$a+$b;  
}*  
  
add( a: 4, b: 5 );
```

This makes it easy to learn Cuneiform. It also makes it straightforward to see what Cuneiform has to offer with only a small time investment.

Automatic Parallelism

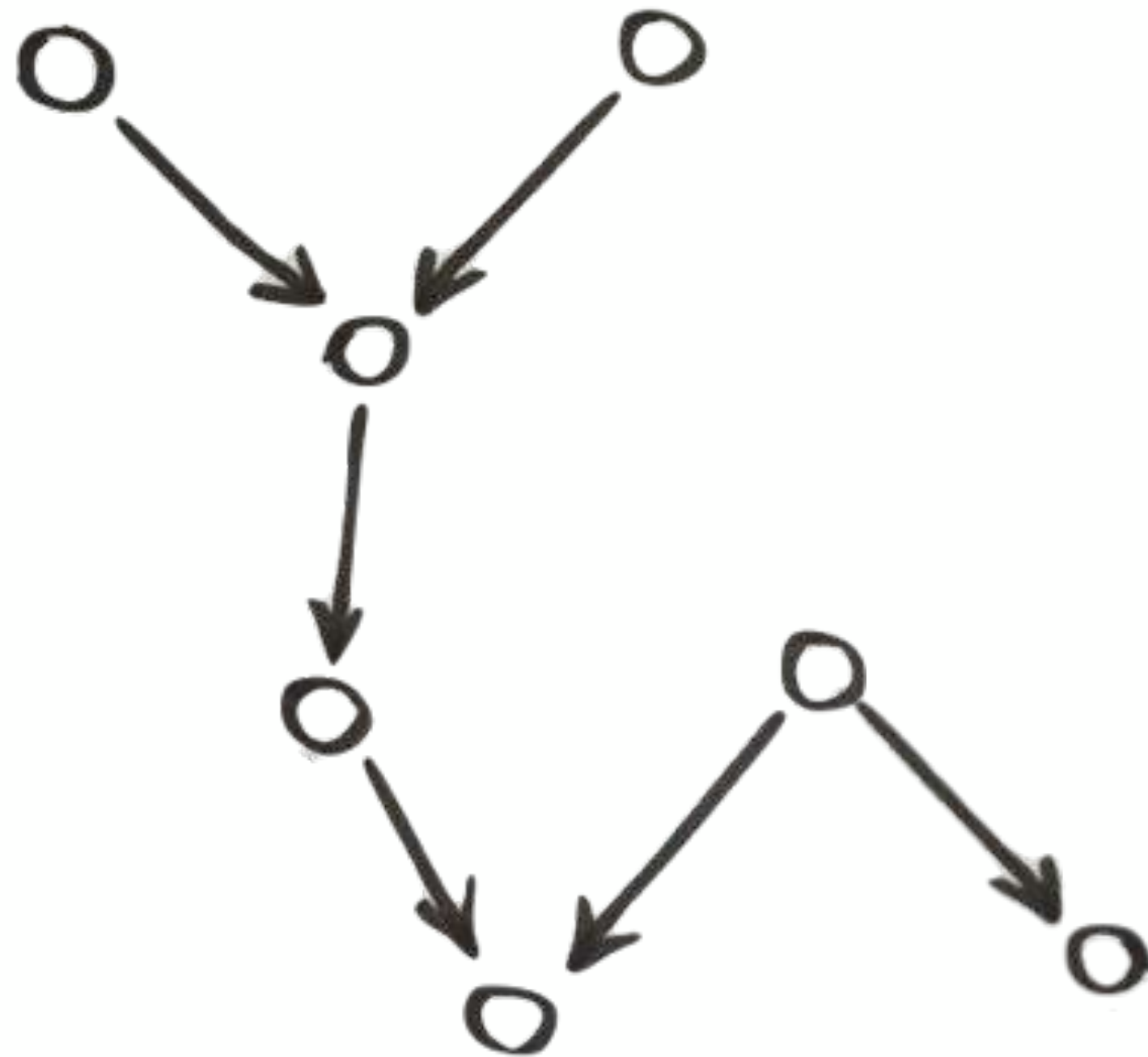
Having to develop operators in an application domain is often hard enough. Additionally having to mine more or potential parallelism is

Take I: Cuneiform backend

- Jörgen Brandt's (@joergenbr)
cuneiform-lang.org
- *erlang* based language to
abstract foreign language
interaction and parallelisation
- Query-based graph of tasks

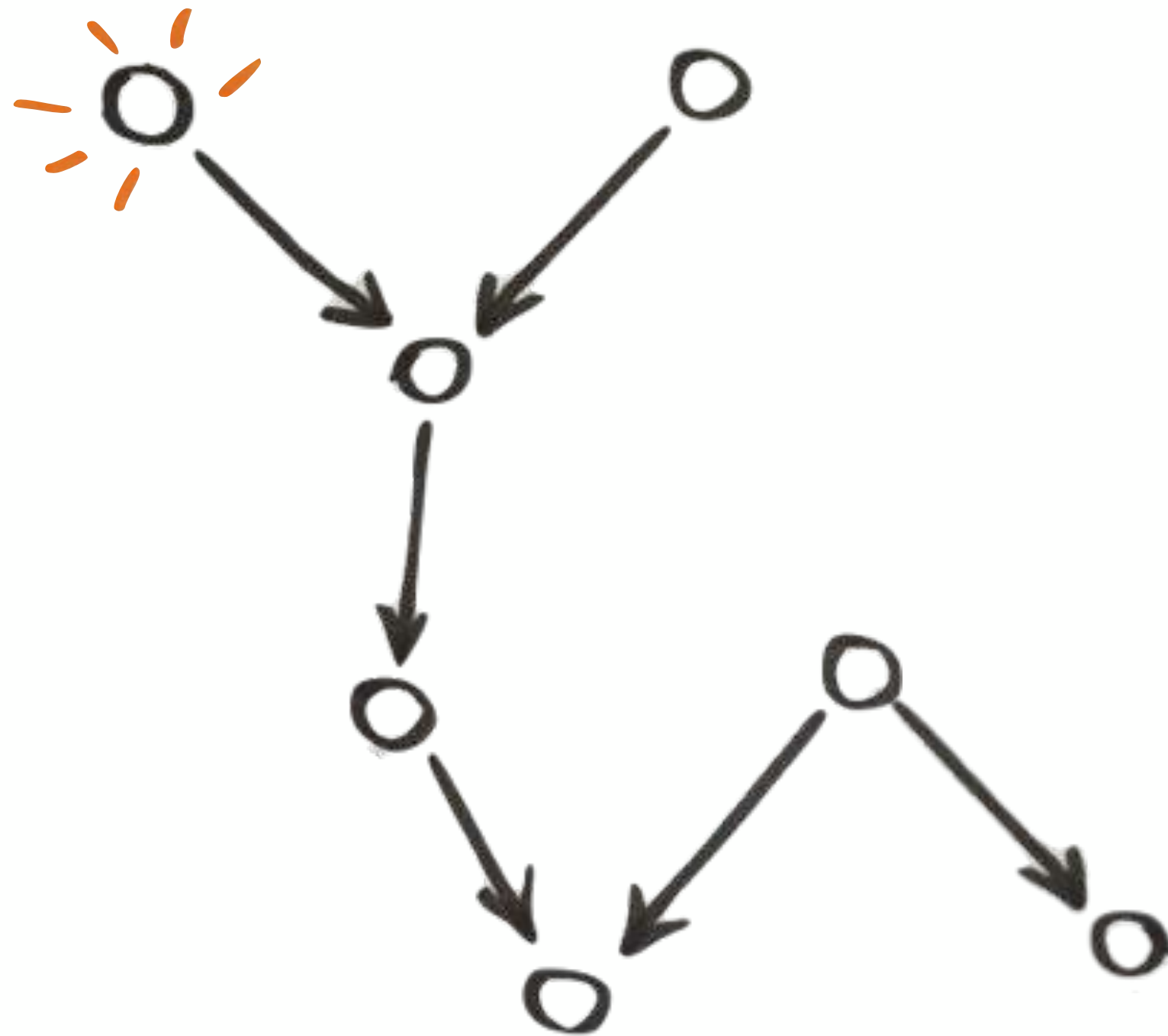
```
deftask etl_op(  
    csv( File )  
    : xml( File ), jar( File ) )  
in Bash *{  
    csv=table.csv  
    java -jar $jar -in $xml -out $csv  
}*  
  
deftask visualize(  
    pdf( File )  
    : csv( File ) )  
in R *{  
    library( my_fancy_r_library )  
    data <- read.csv( csv )  
    img <- visualize.data( data )  
    pdf <- write.pdf( img, "img.pdf" )  
}*  
  
jar = "etl.jar";  
xml = "my_data.xml";  
csv = etl_op( xml: xml, jar: jar );  
pdf = visualize( csv: csv );
```

Take I: Cuneiform backend



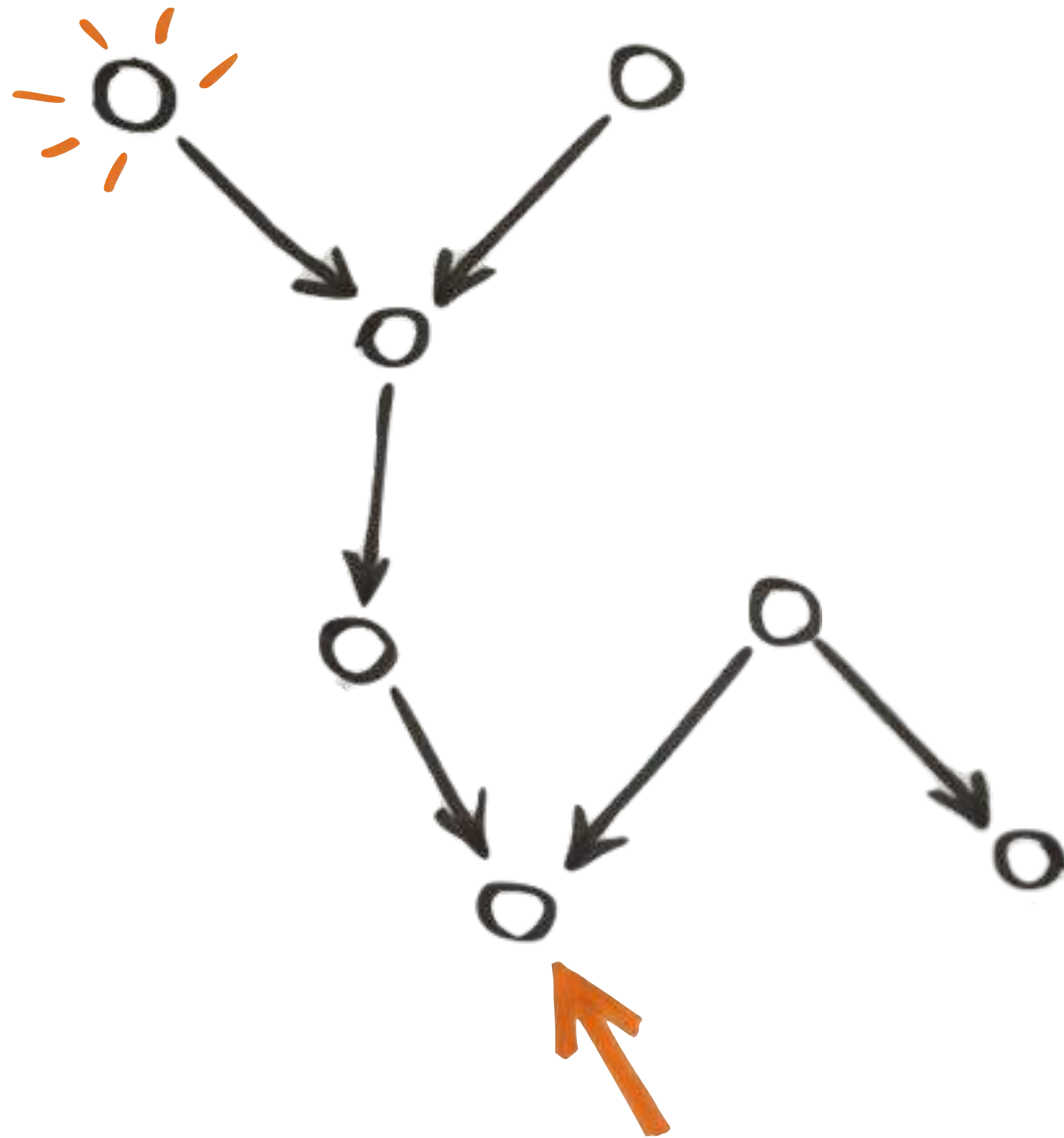
```
deftask etl_op(  
    csv( File )  
    : xml( File ), jar( File ) )  
in Bash *{  
    csv=table.csv  
    java -jar $jar -in $xml -out $csv  
}*  
  
deftask visualize(  
    pdf( File )  
    : csv( File ) )  
in R *{  
    library( my_fancy_r_library )  
    data <- read.csv( csv )  
    img <- visualize.data( data )  
    pdf <- write.pdf( img, "img.pdf" )  
}*  
  
jar = "etl.jar";  
xml = "my_data.xml";  
csv = etl_op( xml: xml, jar: jar );  
pdf = visualize( csv: csv );
```

Take I: Cuneiform backend



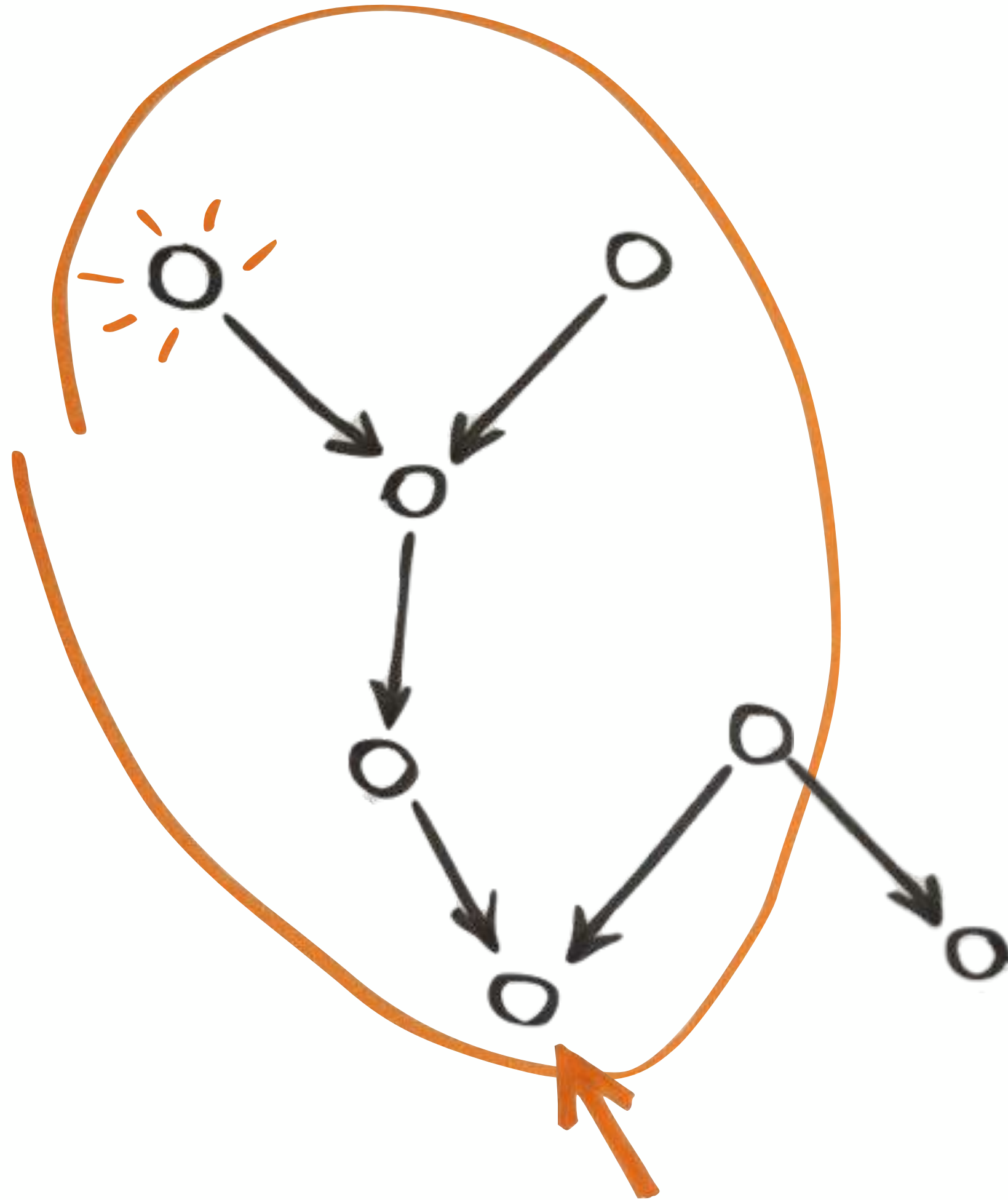
```
deftask etl_op(  
    csv( File )  
    : xml( File ), jar( File ) )  
in Bash *{  
    csv=table.csv  
    java -jar $jar -in $xml -out $csv  
}*  
  
deftask visualize(  
    pdf( File )  
    : csv( File ) )  
in R *{  
    library( my_fancy_r_library )  
    data <- read.csv( csv )  
    img <- visualize.data( data )  
    pdf <- write.pdf( img, "img.pdf" )  
}*  
  
jar = "etl.jar";  
xml = "my_data.xml";  
csv = etl_op( xml: xml, jar: jar );  
pdf = visualize( csv: csv );
```


Take I: Cuneiform backend



```
deftask etl_op(  
    csv( File )  
    : xml( File ), jar( File ) )  
in Bash *{  
    csv=table.csv  
    java -jar $jar -in $xml -out $csv  
}*  
  
deftask visualize(  
    pdf( File )  
    : csv( File ) )  
in R *{  
    library( my_fancy_r_library )  
    data <- read.csv( csv )  
    img <- visualize.data( data )  
    pdf <- write.pdf( img, "img.pdf" )  
}*  
  
jar = "etl.jar";  
xml = "my_data.xml";  
csv = etl_op( xml: xml, jar: jar );  
pdf = visualize( csv: csv );
```

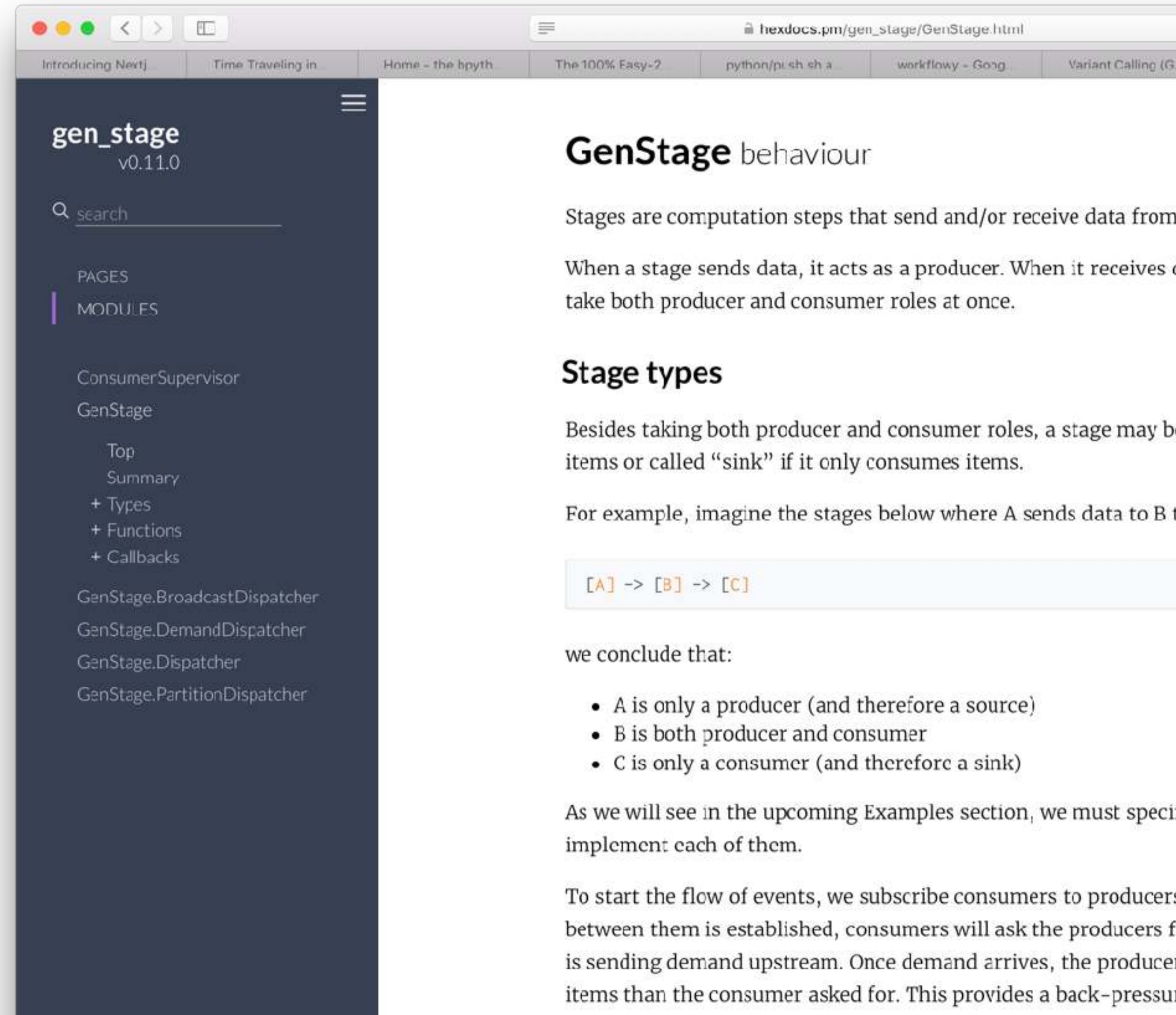

Take I: Cuneiform backend



```
deftask etl_op(  
    csv( File )  
    : xml( File ), jar( File ) )  
in Bash *{  
    csv=table.csv  
    java -jar $jar -in $xml -out $csv  
}*  
  
deftask visualize(  
    pdf( File )  
    : csv( File ) )  
in R *{  
    library( my_fancy_r_library )  
    data <- read.csv( csv )  
    img <- visualize.data( data )  
    pdf <- write.pdf( img, "img.pdf" )  
}*  
  
jar = "etl.jar";  
xml = "my_data.xml";  
csv = etl_op( xml: xml, jar: jar );  
pdf = visualize( csv: csv );
```

Take II: GenStage

- $[A] \rightarrow [B] \rightarrow [C]$
- “thin” layer on top of GenServer
- producers, consumers and producer-consumers
- demand-based batch-processing pipelines
- “rich” stateful subscriptions
- configurable dispatchers



The screenshot shows the GenStage v0.11.0 documentation page. The left sidebar contains a navigation menu with sections for 'PAGES' and 'MODULES'. Under 'MODULES', there is a list of modules including 'ConsumerSupervisor', 'GenStage', and several dispatcher modules. The main content area is titled 'GenStage behaviour' and explains that stages are computation steps that send and/or receive data. It also discusses stage types and provides an example of a pipeline: $[A] \rightarrow [B] \rightarrow [C]$. Below the example, it concludes that A is only a producer, B is both producer and consumer, and C is only a consumer. The page also mentions that specific dispatchers must be implemented for each stage.

GenStage behaviour

Stages are computation steps that send and/or receive data from other stages.

When a stage sends data, it acts as a producer. When it receives data, it acts as a consumer. A stage can take both producer and consumer roles at once.

Stage types

Besides taking both producer and consumer roles, a stage may be a source (only producer) or a sink (only consumer) or called “sink” if it only consumes items.

For example, imagine the stages below where A sends data to B and B sends data to C:

```
[A] -> [B] -> [C]
```

we conclude that:

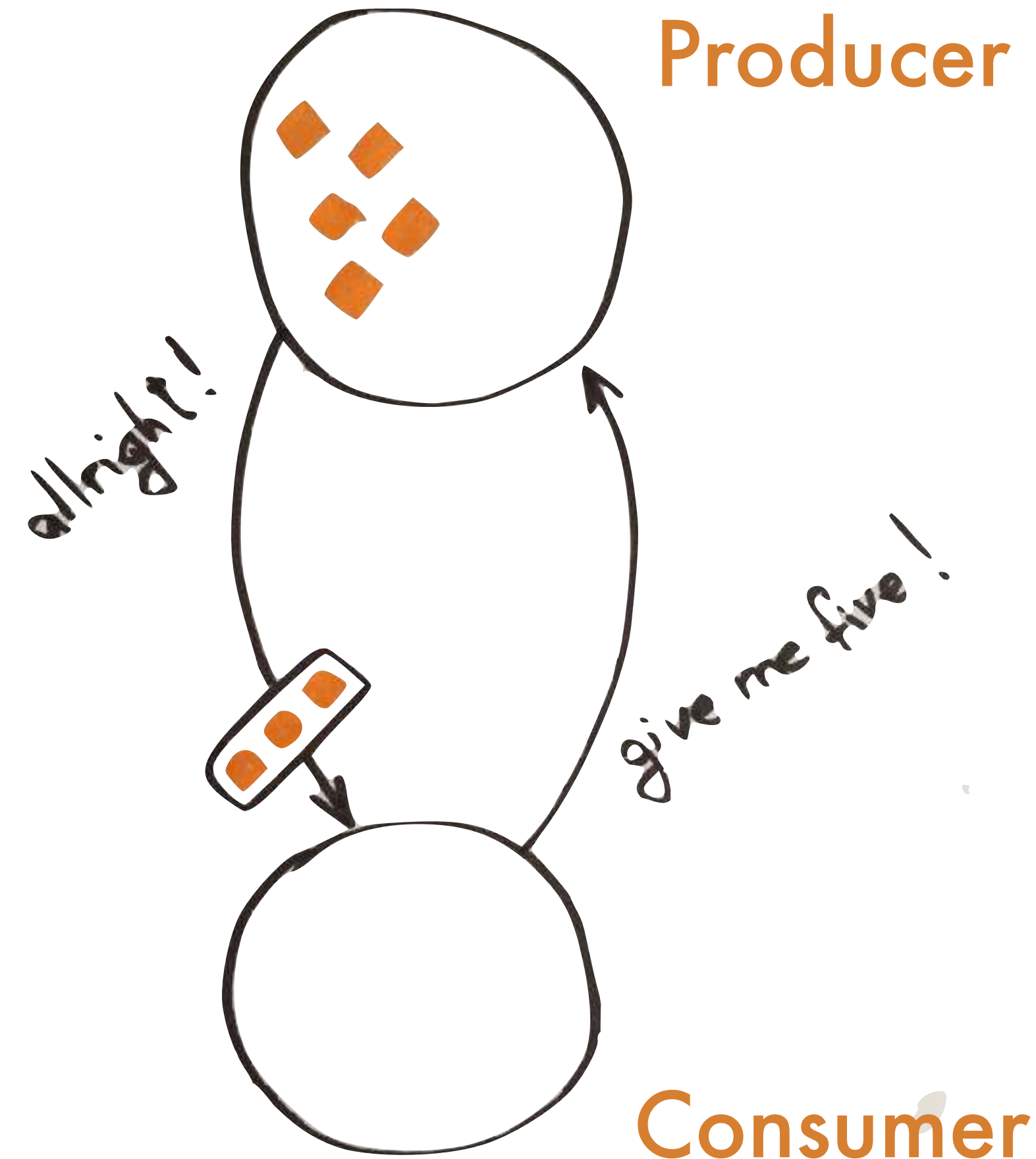
- A is only a producer (and therefore a source)
- B is both producer and consumer
- C is only a consumer (and therefore a sink)

As we will see in the upcoming Examples section, we must specify a dispatcher for each stage to implement each of them.

To start the flow of events, we subscribe consumers to producers. Once a subscription is established, consumers will ask the producers for data. The producer will then send demand upstream. Once demand arrives, the producer will send items to the consumer. This provides a back-pressure mechanism.

Take II: GenStage

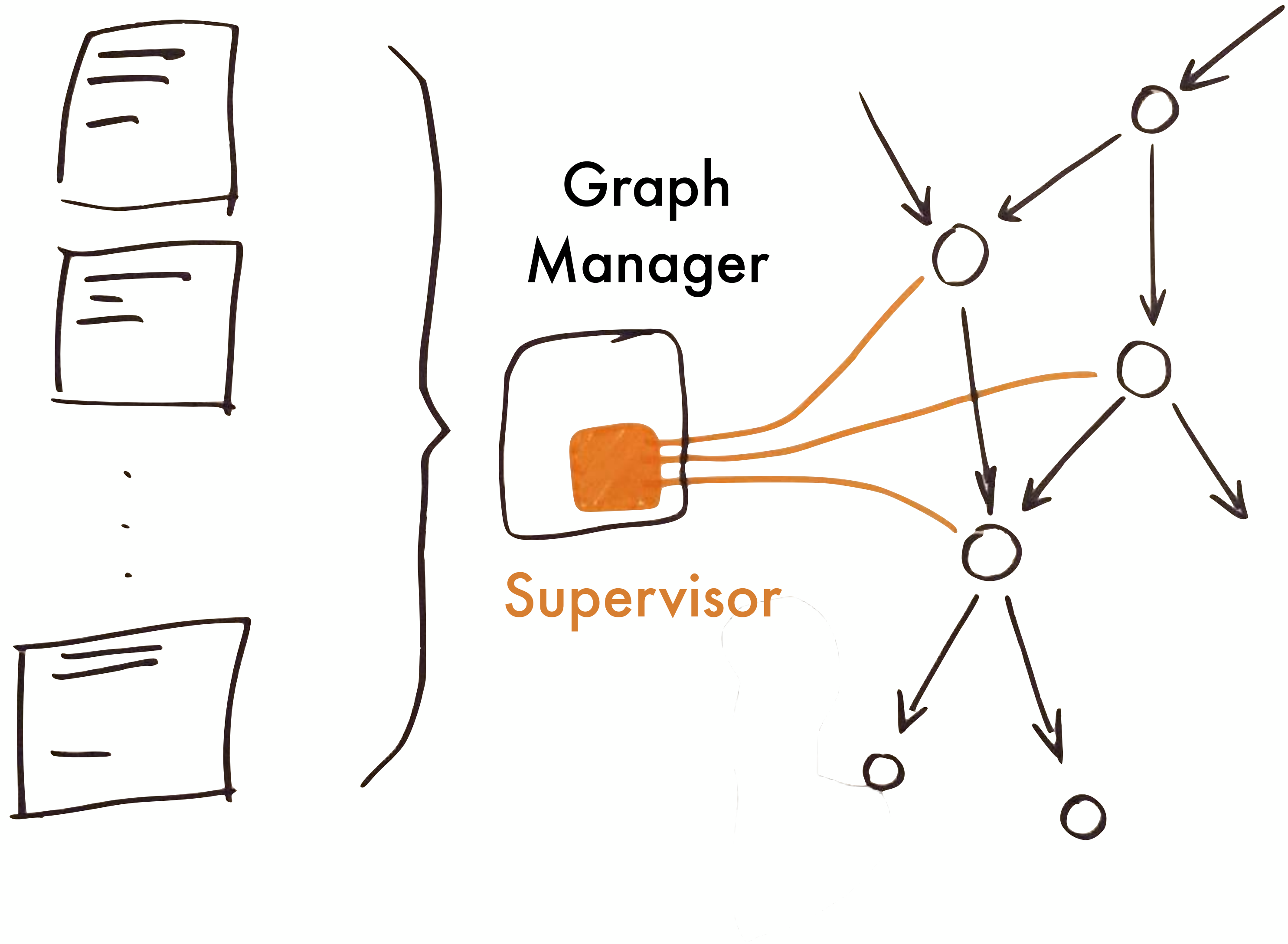
- $[A] \rightarrow [B] \rightarrow [C]$
- “thin” layer on top of GenServer
- producers, consumers and producer-consumers
- demand-based batch-processing pipelines
- “rich” stateful subscriptions
- configurable dispatchers



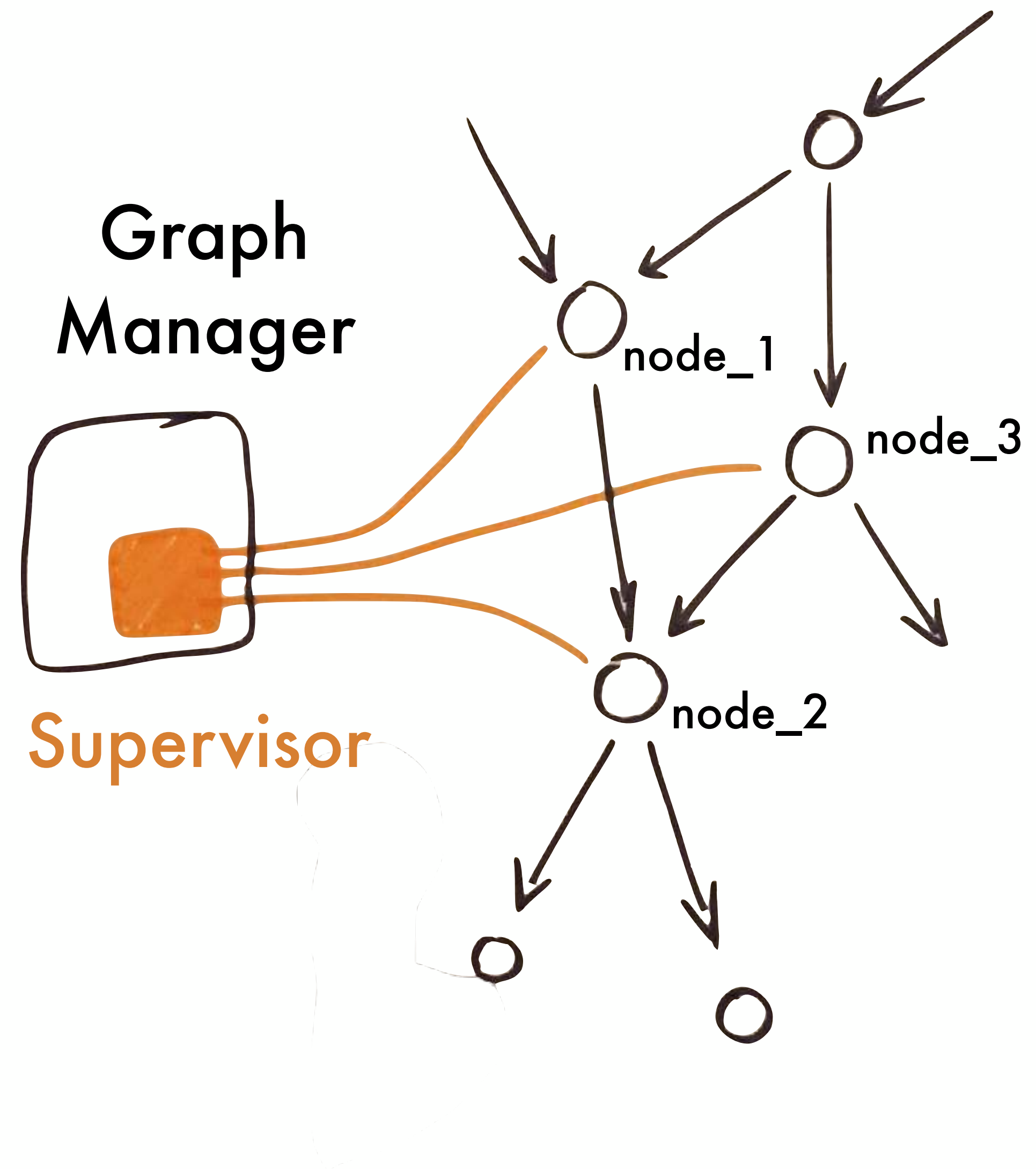
Take II: GenStage

- $[A] \rightarrow [B] \rightarrow [C]$
- “thin” layer on top of GenServer
- producers, consumers and producer-consumers
- demand-based batch-processing pipelines
- “rich” stateful subscriptions
- configurable dispatchers

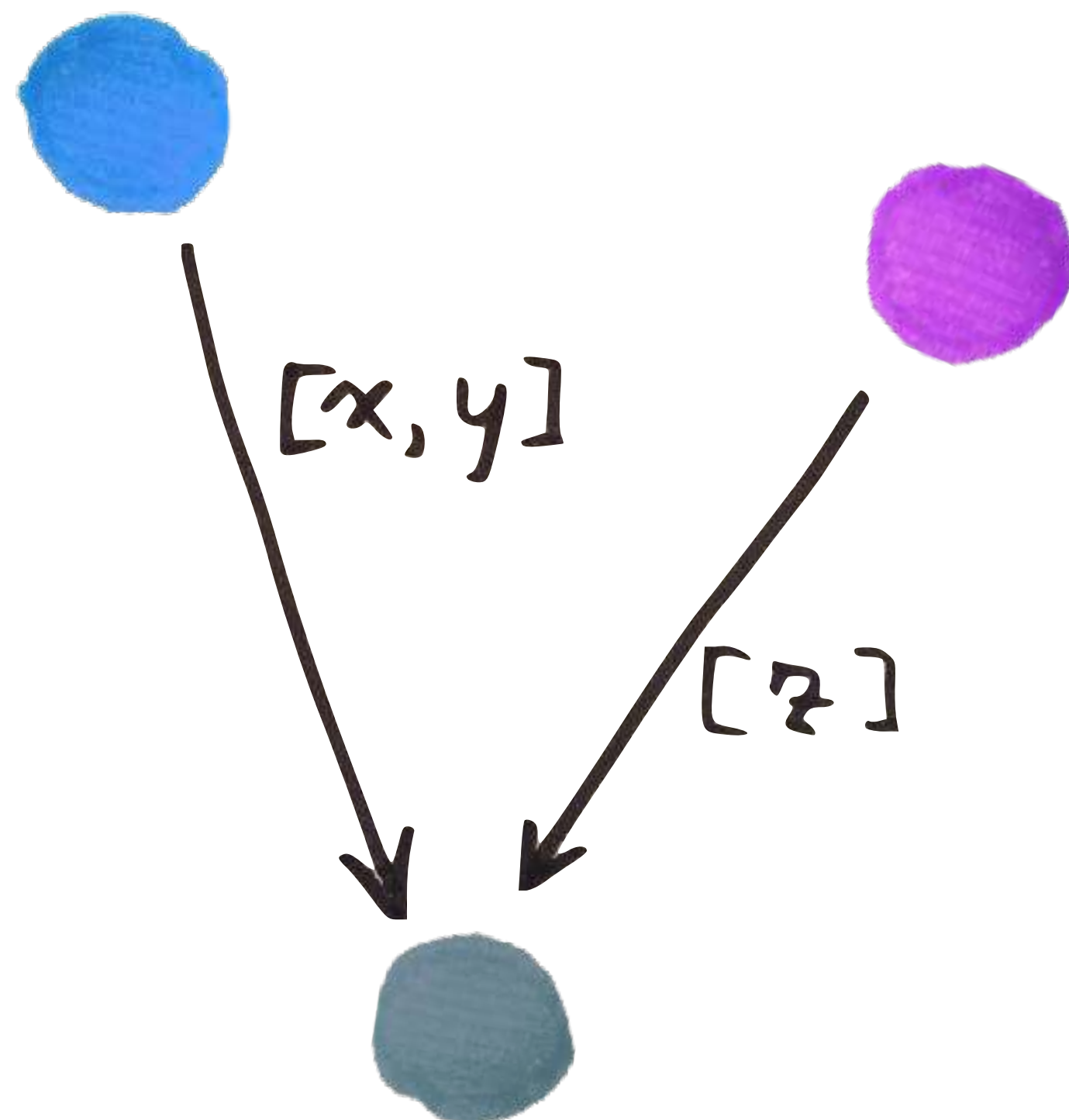




```
"node_1":  
  type: "value"  
  value: 12  
  
"node_2":  
  type: "code-cell"  
  body: "${node_1} + ${node_3}"  
  lang: "julia"  
  dependencies:  
    - id: "node_1"  
    - id: "node_3"  
    refs: []  
  
"node_3":  
  type: "code-cell"  
  body: "3"  
  lang: "python"
```



Requiring Exports



● blue

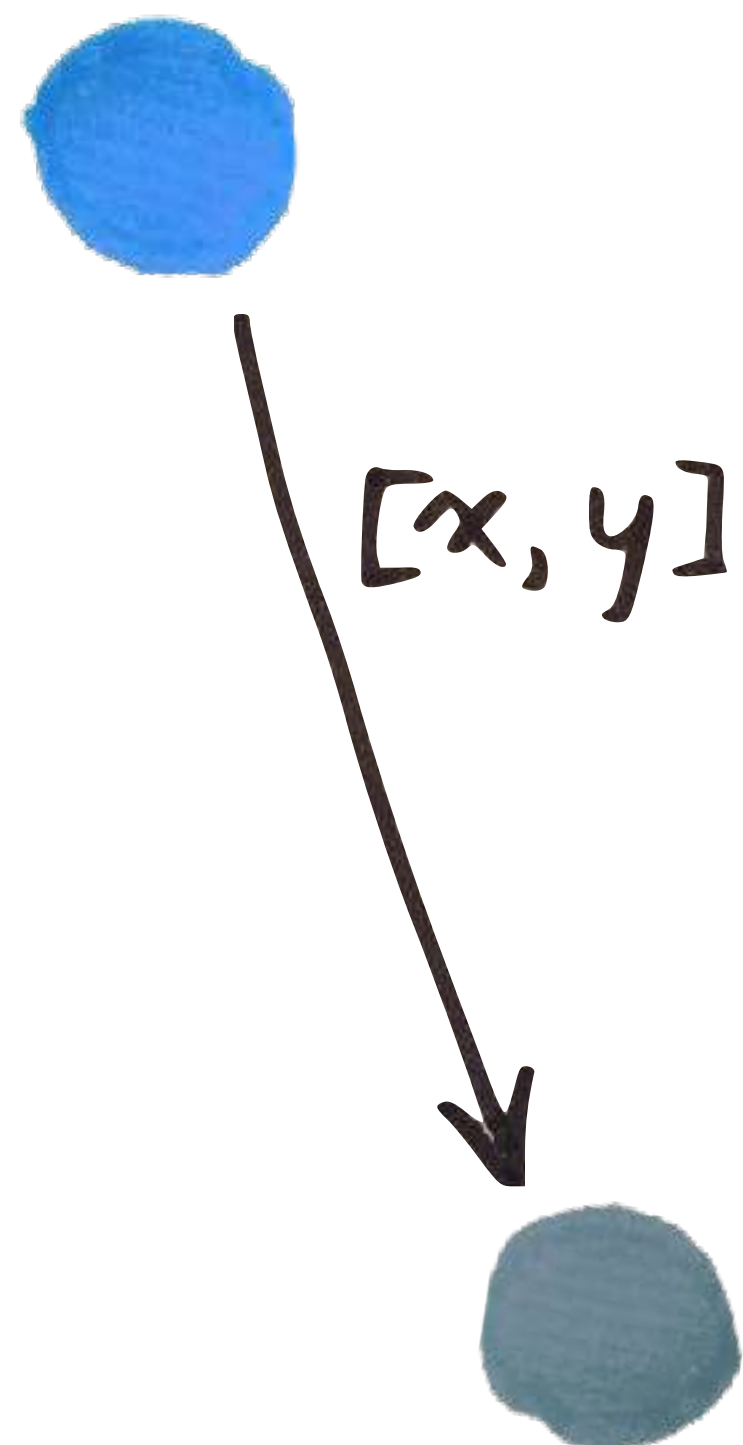
```
x = 1  
y = 2
```

● purple

```
z = [3, 4]
```

● bluegray

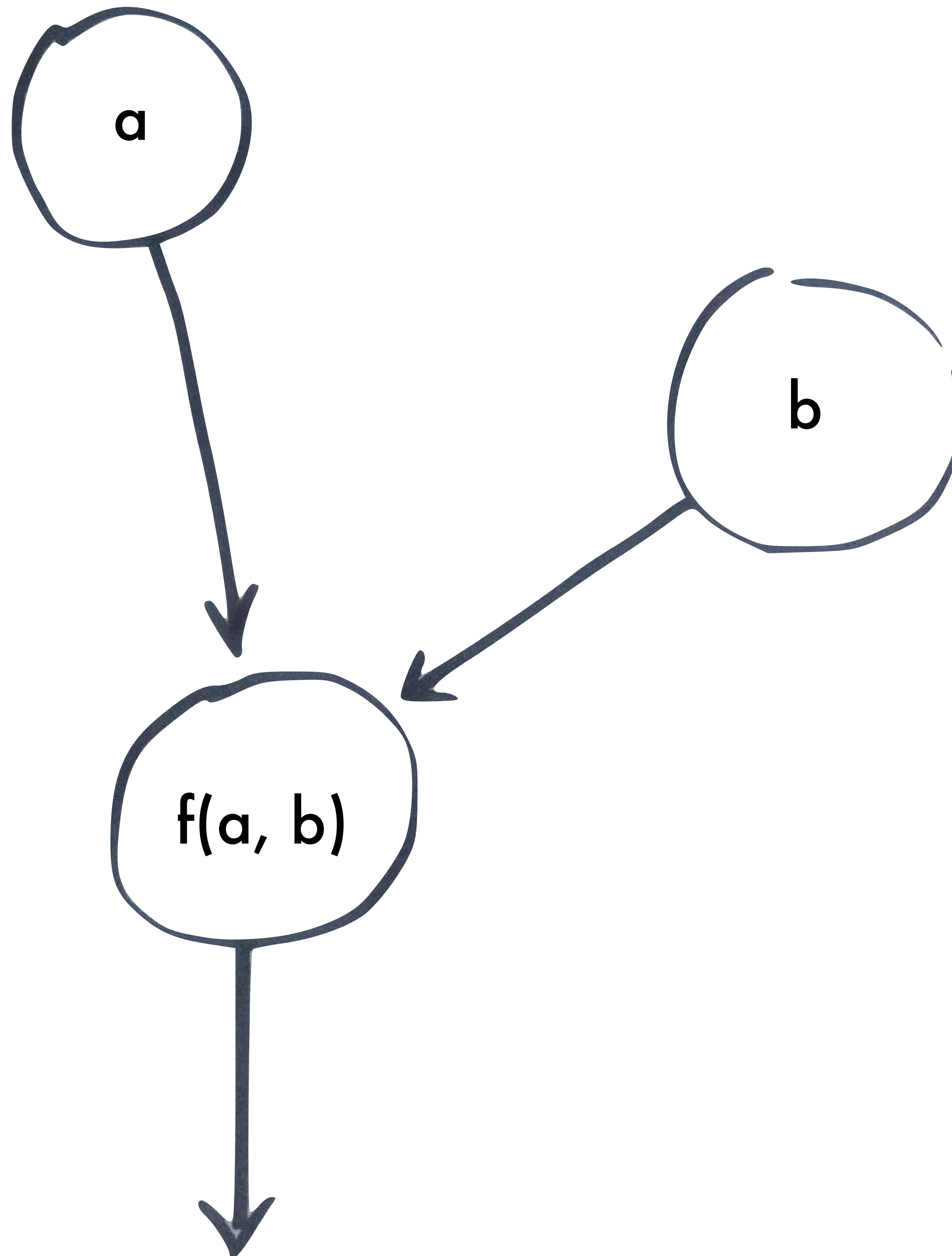
```
out <- c(blue.x, blue.y) + purple.z
```

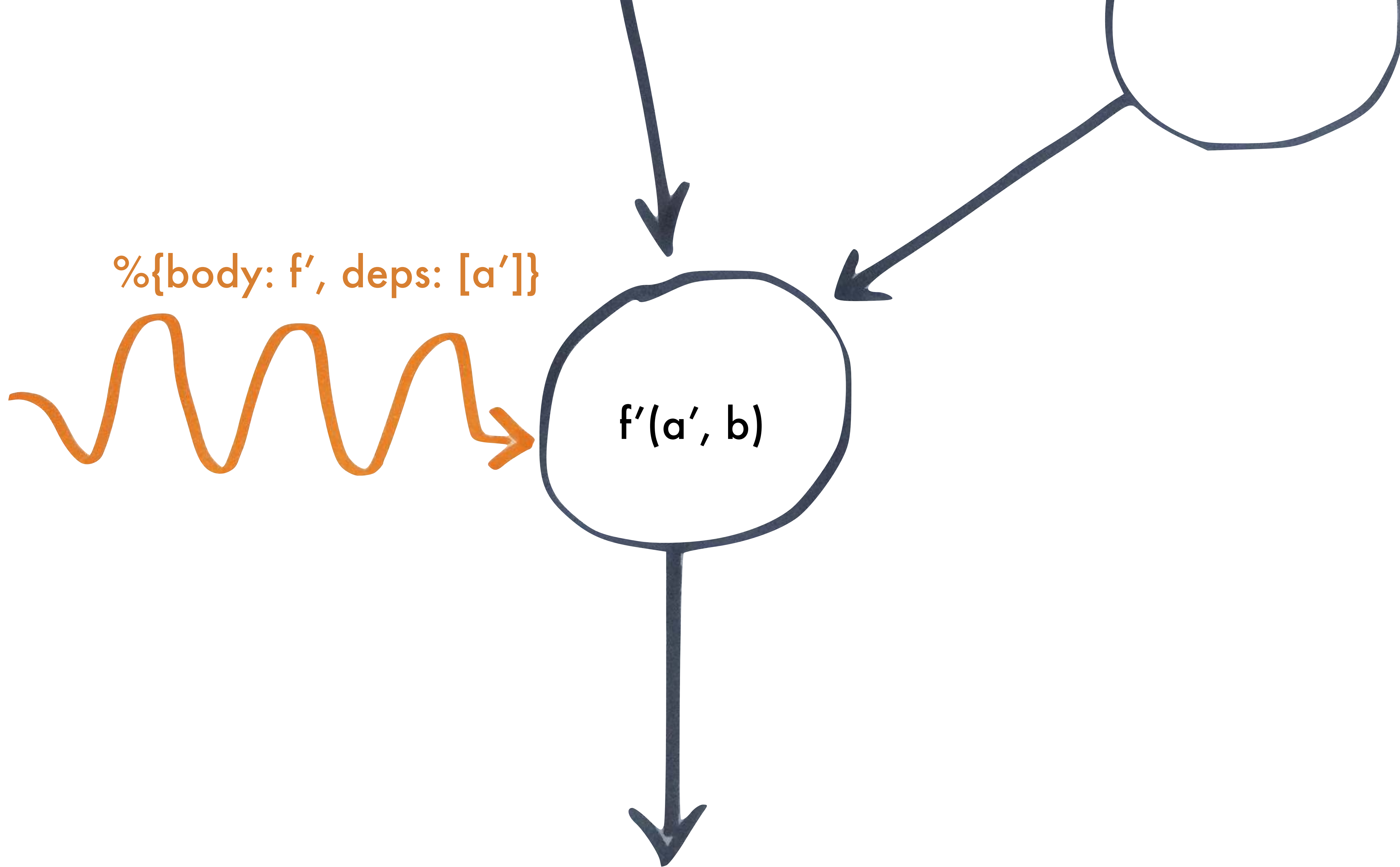



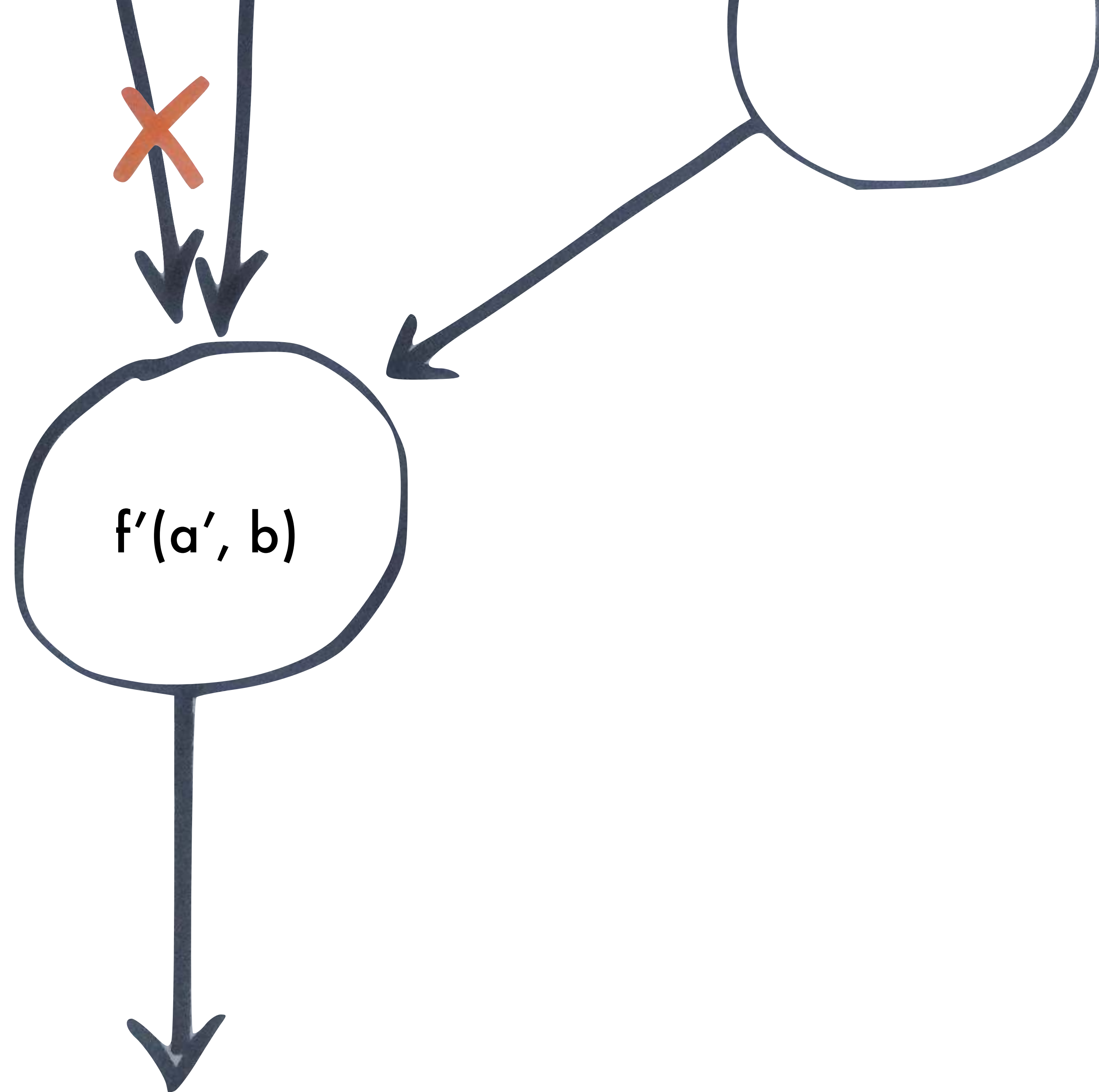
```
GenStage.async_subscribe(grey,  
  to: blue,  
  refs: [x, y])
```

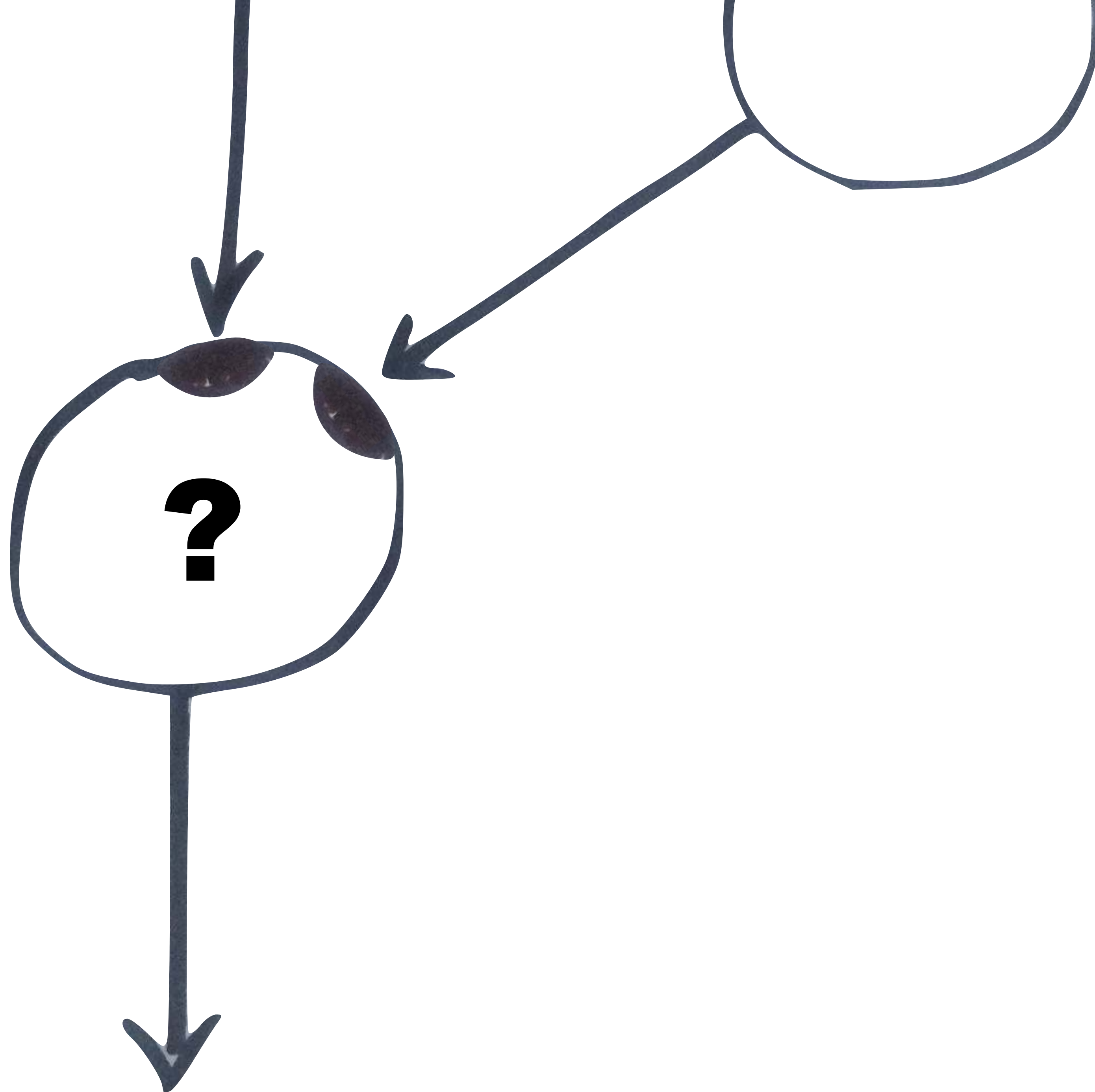
```
handle_subscribe(:consumer,  
  [refs: [x, y], ..., state])
```

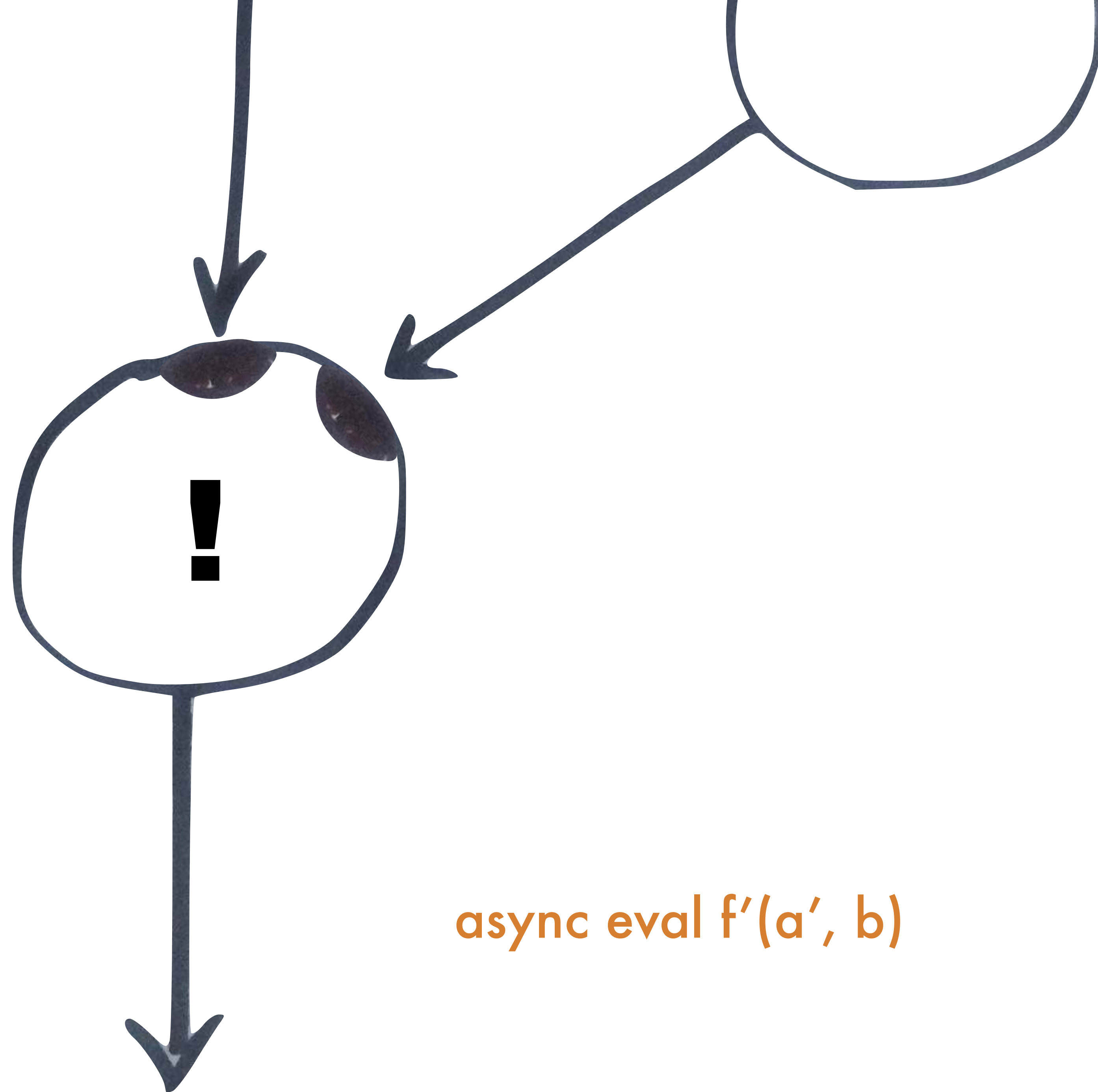
```
handle_subscribe(:producer,  
  [refs: [x, y], ..., state])
```







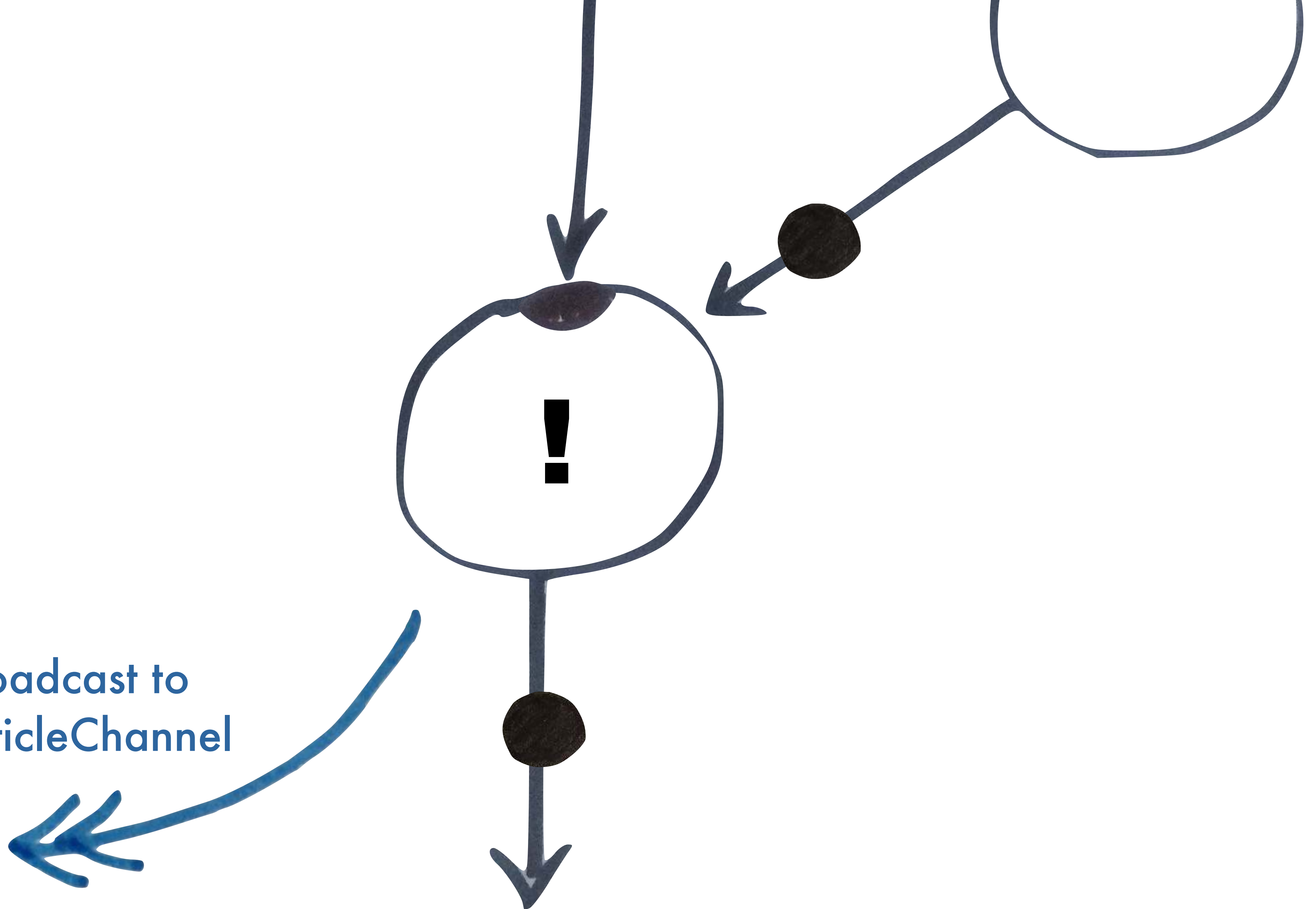


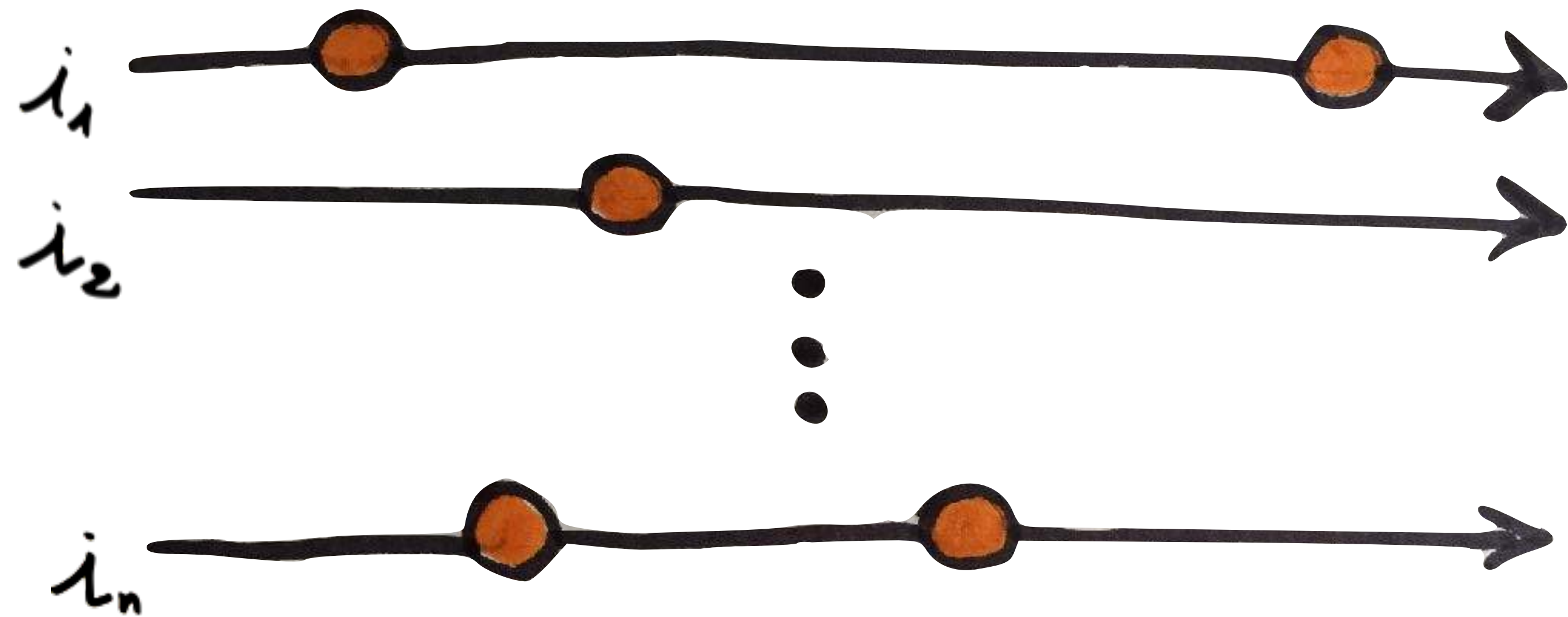
broadcast to
ArticleChannel

$f'(a', b)$

The diagram features a central white circle with a dark blue outline, containing the text $f'(a', b)$. A dark blue arrow points down from the top into the circle. Another dark blue arrow points from the top right towards the circle. Below the circle, a dark blue arrow points down through a solid dark blue circle. A dark blue arrow points from the bottom left towards the bottom of the central circle. A blue arrow points from the bottom left towards the bottom of the central circle, with the text "broadcast to ArticleChannel" next to it.

broadcast to
ArticleChannel





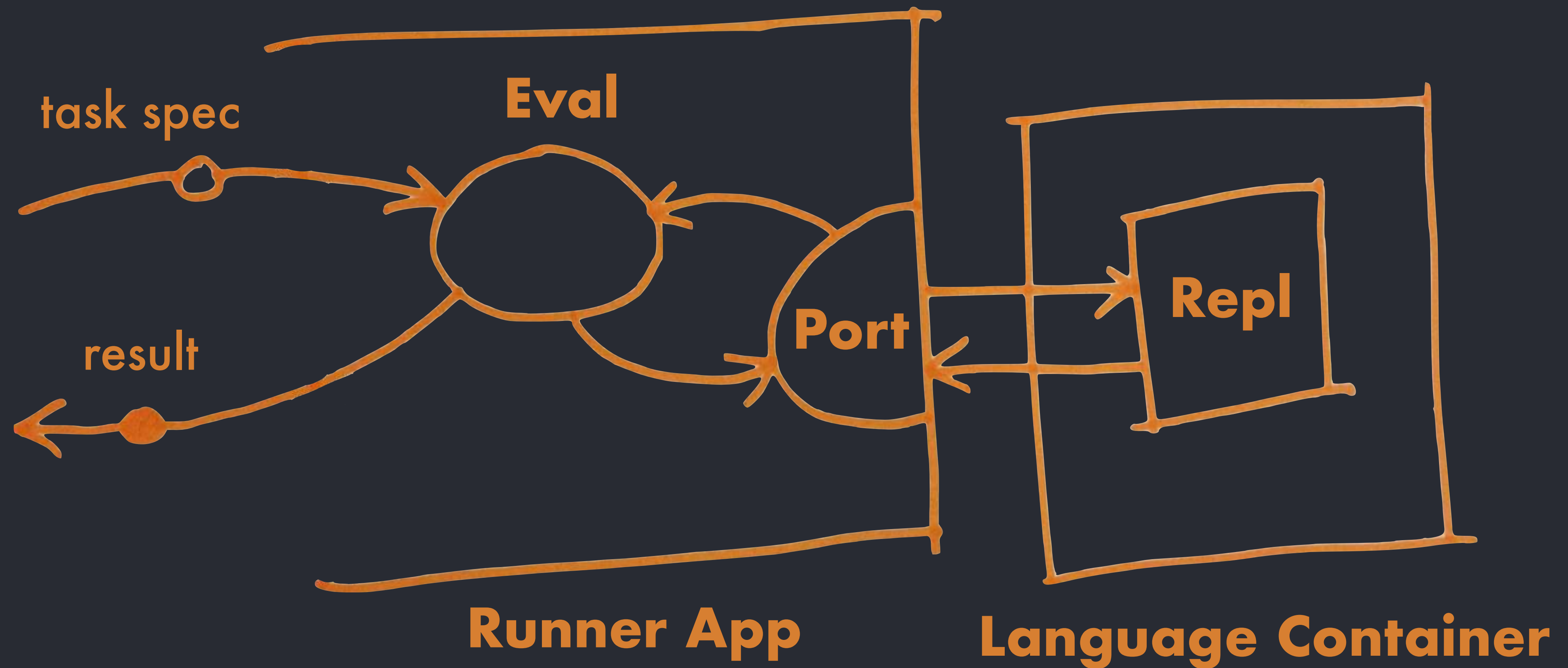
combineLatest ($i_1, \dots, i_n$) $\Rightarrow$ eval(code; $i_1, \dots, i_n$)



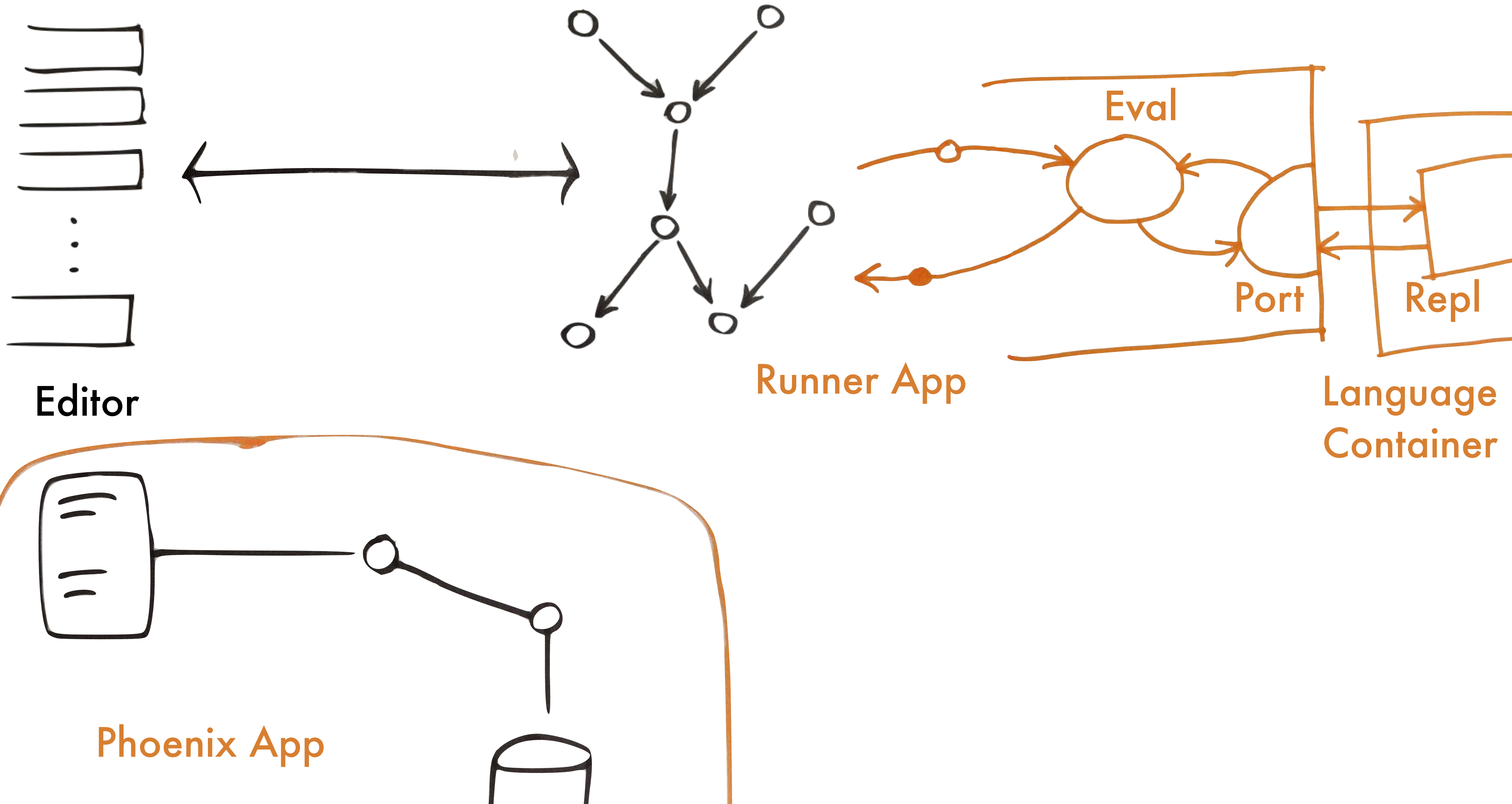
```
def handle_events([%Result{} = res], _from, %CodeCell{}) do
  # ...
  {state, errors} =
    |> add_result(state, res)
    |> trigger?
    |> async_schedule_task()

  {:noreply, errors, state}
end
```

```
def handle_cast(%Result{id: id} = res, %CodeCell{id: id}) do
  # ...
  broadcast(state.topic, "result", res)
  {:noreply, [res], state}
end
```



```
:erlang.open_port(  
  {:spawn_executable, 'docker'},  
  args: ['run', '-i', docker_image, command])
```

Remarks & Thanks

ne>tjournal

@usenextjournal

see you at CurryOn, Barcelona, June 19-20th